

编译原理 · 从零开始的讲义

编译器是怎样炼成的

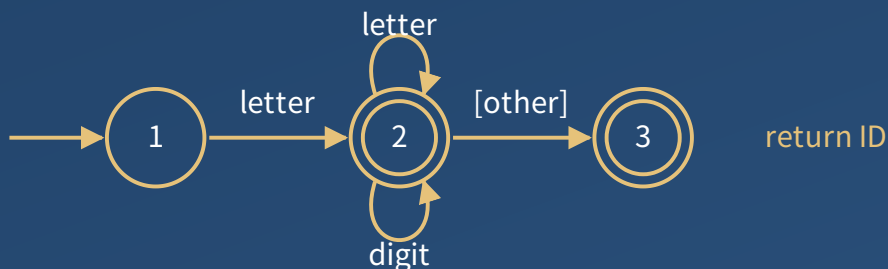
第一部：概览与扫描

Compiler Construction, Part I: Overview & Scanning

以 OSTEP 的方式讲述编译器的第 1、2 章

对话 · 关键问题 · 提示 · 补充 · 图解 · 作业

预备知识 · 编译器概览 · 扫描（正则表达式与有限自动机）



依据哈尔滨工程大学《编译原理》第 1-2 章课件整理编写

参考教材: K. C. Louden, *Compiler Construction: Principles and Practice*

为 iPad Pro 11" 横屏/竖屏阅读排版 · 2026 年 9 月

致读者：关于这本讲义的对话

一段对话

教授： 欢迎来学编译原理。你知道编译器是什么吗？

学生： 呃……就是那个把代码变成程序的东西？我按一下"运行"，它就干活了。

教授： 说得不错，但"变成"这两个字里藏着一整门课。想想看：你写下 `a[index] = 4 + 2`，机器却只认识一串 0 和 1。中间发生了什么？谁把 `index` 这五个字母看成一个整体？谁决定先算加法再赋值？谁把它翻译成机器指令？谁把"4 + 2"直接算成 6 好让程序快一点？

学生： 听起来是一大堆事。

教授： 是一大堆，但每一件都不难。这门课的美妙之处在于：它把一个看起来像魔法的过程，拆成一连串**清清楚楚的小问题**，每个小问题都有优雅的理论 and 朴素的代码。学完之后，你看任何一门编程语言的眼光都会不一样。

学生： 可是我数学一般，也没学过什么"自动机"。

教授： 所以这本讲义从零开始。第 0 章会把你需要的所有预备知识——字符串、集合、树、状态机的直觉——用最平实的话讲一遍。你只要会一点点 C 语言，并且愿意动手画图，就够了。

学生： 那我们从哪里开始？

教授： 从一个问题开始。每一章的核心，都会以一个**"关键问题"**的形式提出来。带着问题读，比背结论有用一百倍。

这本讲义怎么用

这本讲义模仿了《Operating Systems: Three Easy Pieces》（OSTEP）的写法：先提问题，再给答案；先讲直觉，再讲定义；先举例子，再谈一般性。你会在正文里反复见到下面几种"盒子"：

◆ 关键问题：这是本节要解决的核心问题 ◆

每一节围绕一个明确的问题展开。读完这一节，你应该能用自己的话回答它。

提示：可以带走的经验

一条可以反复使用的经验法则、记忆技巧或者常见陷阱。

补充：更宽一点的视野

历史、轶事、与其他课程的联系，或是课件之外但有助于理解的内容。跳过不影响主线，读了会更有趣。

此外，每章开头有一段**对话**，帮你进入情境；章末有**小结和作业**（课件里布置的原题，附录 B 给出提示与参考思路）。所有的图都是重新绘制的，尽量让每个箭头、每个圆圈都有解释。

在 iPad 上阅读时，建议横向或纵向单页显示，用 Apple Pencil 直接在自动机图上"走一遍"输入串——这是学会本课最快的方式。

内容一览

- **第 0 章 预备知识**：机器如何理解程序、字符串与语言、集合运算、树、状态机的直觉、读懂课件代码所需的一点 C。
- **第 1 章 编译器概览**：编译器是什么、它的亲戚们、翻译的六个阶段、主要数据结构、前端与后端、自举与移植、TINY 语言与 TM 机器。
- **第 2 章 扫描**：记号、正则表达式、DFA 与 NFA、把自动机变成代码的三种方法。
- **附录**：中英术语对照、作业提示与参考思路、自测清单。

目录

致读者：关于这本讲义的对话	ii
---------------------	----

第 0 章 预备知识：在编译器之前	6
-------------------------	---

0.1 机器只认识数字	6
0.2 程序、语言与"翻译"	7
0.3 字符、字符串与字母表	8
0.4 语言就是字符串的集合	9
0.5 树：把结构画出来	10
0.6 状态机的直觉	11
0.7 读懂课件代码所需的一点 C	13
0.8 符号速查与小结	14

第 1 章 编译器概览	16
-------------------	----

1.1 为什么需要编译器	17
1.2 编译器的亲戚们	20
1.3 翻译过程：六个阶段	22
1.4 编译器中的主要数据结构	27
1.5 编译器结构中的其他问题	29
1.6 自举与移植	31
1.7 TINY 语言与 TM 机器	36
小结与作业	39

第 2 章 扫描：从字符到记号	41
-----------------------	----

2.1 扫描的任务	42
2.2 正则表达式：描述模式	44
2.2.1 基本定义与三种运算	44
2.2.4 五个例题，一次学会写正则	46
2.2.6 描述真实的记号	48
2.3 有限自动机：识别模式	51
2.3.2 确定性有限自动机 (DFA)	51
2.3.4 合并多个 DFA	55

2.3.5 ε -转换与非确定性有限自动机 (NFA)	56
2.4 把自动机变成代码	59
小结与作业	62
附录 A 术语中英对照表	63
附录 B 作业提示与参考思路	68
附录 C 自测清单	72

第 0 章

预备知识：在编译器之前

Prerequisites: Before the Compiler

这一章不讲编译器。它讲的是编译器这门课默认你"已经知道"、但很少有人真正教过的东西：机器怎样理解程序，字符串和语言的精确含义，树，以及状态机的直觉。如果你觉得自己基础扎实，可以快速浏览；但请至少读一读 0.3、0.4 和 0.6——第 2 章几乎每一页都建立在它们之上。

对话：为什么要先讲"预备知识"？

学生： 课件第一页就是"编译器是把一种语言翻译成另一种语言的程序"，我觉得我听得懂啊，为什么还要预备知识？

教授： 你当然听得懂这句话。问题是第二章：突然冒出来 Σ 、 $L(r)$ 、 $(a|c)^*b(a|c)^*$ 、 $T: S \times \Sigma \rightarrow S \cdots$ 很多同学就是在这里掉队的。

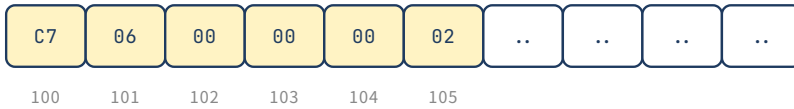
学生： 确实，那些符号看起来像另一门课。

教授： 它们其实只是在**精确地**说一些非常朴素的事：什么是"一串字符"、什么是"一类字符串"、什么是"一步一步地读"。这一章我们就把这些朴素的事说清楚。学会之后，第二章的符号就只是速记，而不是障碍。

0.1 机器只认识数字

先从最底层说起。一台计算机的内存，可以想成一排**编了号的小格子**，每个格子装一个 8 位的二进制数（一个**字节**，byte）。格子的编号叫**地址**（address）。CPU 做的事情极其单调：从某个地址取出一个数，把它当成"指令"来解释，执行，再取下一个。

内存：编了号的字节格子（地址从左到右递增）



黄色的 6 个字节合起来是一条 8086 指令：把数字 2 存到地址 0000 处

图 0.1 内存是一排编号的字节；指令本身也只是存放在内存里的数字。课件中的例子 `C7 06 0000 0002` 就是这样一条"把 2 放进地址 0000"的机器指令。

这就是**机器语言**（machine language）：程序是一串数字，每个数字代表一个动作。最早的程序员真的就是这样写程序的——查表、手工编码、用开关或纸带输入。这样写程序**费时、乏味、容易出错**，而且换一种 CPU，所有数字的含义都变了。

于是人们想：能不能给每个数字起个名字？把 `C7 06` 叫 `MOV`，把地址 0000 叫 `X`，那条指令就成了 `MOV X, 2`。这就是**汇编语言**（assembly language）。但是名字和数字之间的对应，总得有人来做——这个"有人"，就是历史上第一批**翻译程序**：汇编器。

提示：整门课只回答一个问题

"人喜欢用符号和结构来表达想法，机器只认识数字。谁来做中间的翻译？怎么做？"记住这句话，第 1 章的所有内容（编译器、汇编器、解释器、连接器……）都是这个问题在不同层次上的答案。

0.2 程序、语言与"翻译"

在这门课里，我们会同时谈论好几种"语言"，先把它们分清楚：

- **源语言**（source language）：你写程序用的语言，如 C、Pascal，或者课程里的小语言 TINY。
- **目标语言**（target language）：翻译的结果，通常是某台机器的机器语言或汇编语言。

- **实现语言** (implementation / host language)：编译器本身是用什么语言写的。这一点在 1.6 节讨论"自举"时会变得非常有趣。

"翻译"这个词很恰当，但有一个重要的差别：把中文翻成英文，允许一点意译；把 C 翻成机器码，**意思必须一字不差**。编译器输出的目标程序必须与源程序**等价** (equivalent) —— 对任何输入，行为都一样。这条要求贯穿全课：无论做多少"优化"，都不能改变程序的意思。

另一个常见的误解是把"编译"和"运行"混为一谈。编译器只负责**翻译**；翻译完成后，得到的目标程序才被**执行**。它们发生在不同的时间，术语上叫**编译时** (compile time) 和**运行时** (run time)。编译器报告的错误——比如少了个分号、变量没声明——都是编译时错误，程序还一步都没有跑。

0.3 字符、字符串与字母表

编译器读进来的，是一个文本文件：一长串**字符** (character)。第 2 章的全部内容，就是"怎样把一串字符切成有意义的单元"。所以我们需要一套精确谈论"字符串"的语言。这一节的每个定义都很简单，但请务必读慢一点，因为第 2 章会假设你对它们熟悉到不假思索。

字母表 Σ

一个**字母表** (alphabet) 就是一个**有限的字符集合**，习惯上记作 Σ (希腊字母 Sigma)。例如 $\Sigma = \{a, b, c\}$ ，或者 $\Sigma = \{0, 1\}$ ，或者 $\Sigma =$ 全部 ASCII 字符。字母表里的成员叫**符号** (symbol)。

字符串

一个**字符串** (string) 是从 Σ 中取出的符号排成的**有限序列**。abc、aab、b 都是 $\{a, b, c\}$ 上的字符串。字符串的**长度**就是它包含的符号个数。

有一个特殊的字符串，长度为 0，什么符号都没有，叫**空串** (empty string)，记作 ε (希腊字母 epsilon)。它和"空格"不是一回事：空格是一个真实

的字符，而 ϵ 是"没有字符"。很多同学第一次见到 ϵ 时觉得它多余——但你马上会看到，它是让很多定义变得干净的关键。

连接

把两个字符串首尾相接，叫**连接**（concatenation）。`ab` 连接 `c` 得到 `abc`。连接不满足交换律（`ab` $\neq$ `ba`），但 ϵ 是它的"单位元"：任何字符串连接 ϵ ，还是它自己。就像乘法里的 1。

提示： ϵ 就像乘法里的 1

$x \cdot 1 = x$ ， $s \cdot \epsilon = s$ 。凡是遇到"某个东西可以出现、也可以不出现"的情形，"不出现"这个选项就用 ϵ 来表示。第 2 章的 **(+|-| ϵ)** 就是这个意思：正号、负号，或者什么都没有。

0.4 语言就是字符串的集合

◆ 关键问题：如何精确地说出"哪些字符串是合法的" ◆

"标识符"是什么？"数字常量"长什么样？直觉上你都知道，但计算机需要一个精确的定义。我们需要一种既能描述**无穷**多个字符串、本身又是**有限**的说法。

在形式语言理论里，"语言"这个词有一个出乎意料地简单的定义：

一个语言（language）就是一个字符串的集合。

没有别的了。`{a, ab, abb}` 是一个语言（只有三个字符串）。"所有 C 语言的合法标识符"是一个语言（无穷多个字符串）。"所有语法正确的 C 程序"也是一个语言（每个程序本身就是一个很长的字符串）。所以说，判断一个字符串是不是合法标识符，就是在问："它属不属于这个集合？"

集合可以是无穷的，但我们写在纸上的**描述**必须是有限的。怎么用有限的规则描述无限的集合？这正是第 2 章的主题——**正则表达式**是这样的一套记法。为了铺

路，先看三种基本的"集合运算"，它们正好对应正则表达式的三种运算。设 A 和 B 是两个语言：

运算	记法	含义	例子 ($A = \{a, ab\}$, $B = \{c, \epsilon\}$)
并	$A \cup B$	属于 A 或属于 B 的字符串	$\{a, ab, c, \epsilon\}$
连接	AB	前半段取自 A 、后半段取自 B 的所有拼接	$\{ac, a, abc, ab\}$
闭包	A^*	把 A 中的字符串任意个 (含 0 个) 首尾相接	$\{\epsilon, a, ab, aa, aab, aba, abab, \dots\}$

请注意闭包的两个要点：第一，"0 个"也算，所以 ϵ 总是属于 A^* ；第二，每次可以取不同的成员， $aab = a \cdot ab$ 就是这样得到的。星号 $*$ 叫做 **Kleene 星号**，第 2 章会大量使用。

补充： Σ^* 是"所有可能的字符串"

把闭包用在字母表本身上： Σ^* 就是 Σ 上所有有限长度的字符串组成的集合 (包括 ϵ)。因此任何一个 Σ 上的语言，都是 Σ^* 的一个子集。"语言 $\subseteq \Sigma^*$ "这个式子你会在教材里经常见到，它只是在说"语言是某些字符串"。

0.5 树：把结构画出来

字符串是一维的，一个符号跟着一个符号。但程序有**结构**： $4 + 2 * 3$ 里，乘法要先算，所以 $2 * 3$ 是一个整体，然后才和 4 相加。一维的字符串没法直接表达"谁包住谁"，我们需要**树** (tree)。

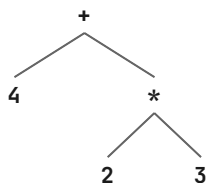


图 0.2 表达式 $4 + 2 * 3$ 的树。树天然地表达了“先算什么、后算什么”：要算根节点 $+$ ，先得算出两个孩子；要算 $*$ ，先得算出 2 和 3。

几个术语，都很直观：最上面的节点叫**根**（root）；一个节点下面挂着的叫它的**孩子**（children），它则是孩子们的**父亲**（parent）；没有孩子的节点叫**叶子**（leaf）；从某个节点出发、连同它下面的一切，叫一棵**子树**（subtree）。

树在编译器里无处不在：第 1 章的分析树、语法树、注释树，都是树。它们在内存里的表示也很朴素——每个节点是一个 C 结构体，里面有若干指向孩子的指针（1.4 节会具体讲）。

树还带来一种思考方式：**递归**。“计算一棵树的值”可以定义为：“如果是叶子，直接返回它的数；否则先计算左右子树的值，再做运算。”这个定义在描述自己的时候用到了自己，但因为树是有限的，它总会在叶子处停下来。编译器里大量的算法都是这个形状。

0.6 状态机的直觉

◆ 关键问题：怎样“一步一步”地读一个字符串 ◆

扫描程序每次只看一个字符。看了 `i`，再看 `f`，它怎么知道这是关键字 `if` 还是标识符 `ifx` 的开头？它必须记住到目前为止看到了什么，但又不能什么都记（那样太慢、太占内存）。

想象一台最简单的自动门禁闸机：它只有两种情况——**锁着**和**开着**。投币，它从“锁着”变成“开着”；推一下，它从“开着”变回“锁着”。它不记得你投过几次币、几

点来的，它只记得**当前处于哪种情况**。这种"情况"就叫**状态**（state），投币和推门叫**输入**（input），从一种情况到另一种情况叫**转换**（transition）。

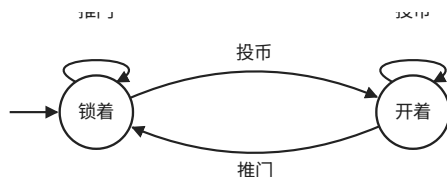


图 0.3 一台闸机的状态机。圆圈是状态，箭头是转换，箭头上写的是触发它的输入。没有来源的箭头指出**起始状态**。

这种"只记住当前处于哪个状态、根据输入切换状态"的机器，叫**有限状态机**（finite-state machine），第 2 章叫它**有限自动机**（finite automaton）。"有限"是关键：状态的个数是固定的，不随输入变长而增加。这带来两个巨大的好处——**速度极快**（每读一个字符只做一次查表）、**内存极省**（只存一个"当前状态"）。这正是扫描程序需要的。

现在把它用在字符串上。假设我们想识别"以字母开头、后面跟任意多个字母或数字"的标识符。我们需要两个状态：状态 1 是"还没读到任何东西"，状态 2 是"已经读到一个合法的开头，后面可以继续"。读到字母时从 1 到 2；在 2 读到字母或数字时留在 2。当输入结束时，如果停在状态 2，就说这个字符串**被接受了**。第 2 章会把这个想法完全形式化，你会发现所有的形式化都只是把图 0.3 这样的图用集合和函数写下来而已。

提示：手指走图

学自动机最有效的方法就是用手指（或 Apple Pencil）在图上走：从起始状态出发，读一个字符，沿着标着这个字符的箭头走到下一个状态；读完后看看停在哪。请对图 0.3 试一下输入序列"投币、推门、推门、投币"。

0.7 读懂课件代码所需的一点 C

课件里的代码不多，但用到几个 C 语言特性，这里做个速览，保证你在第 1、2 章遇到时不会卡住。

枚举类型 enum

```
typedef enum { IF, THEN, ELSE, PLUS, MINUS, NUM, ID } TokenType;
```

枚举就是"给一组整数常量起名字"。IF 其实是 0，THEN 是 1，依此类推。编译器用它来表示"这是哪一类记号"，比用字符串比较快得多，也不容易写错。

结构体 struct

```
typedef struct {  
    TokenType tokenval; /* 记号的类别 */  
    char * stringval; /* 原始的字符串，如 "index" */  
    int numval; /* 如果是数字，它的数值 */  
} TokenRecord;
```

结构体把几个相关的值打包成一个。char * 是"指向字符的指针"，在 C 里就是字符串。第 2 章的"记号记录"就是这样一个结构体。

循环里的 switch

```
state = 1;  
while (state == 1 || state == 2) {  
    switch (state) {  
        case 1: /* 根据下一个字符决定去哪里 */ break;  
        case 2: /* ... */ break;  
    }  
}
```

`switch` 根据一个整数的值跳到对应的 `case`；`break` 跳出 `switch`。"外层 `while`、内层 `switch(state)`"是把状态机写成代码的经典模样，2.4 节会详细分析。

二维数组当作表

```
int T[NSTATES][NCHARS];      /* T[state][ch] = 下一个状态 */
newstate = T[state][ch];
```

一张"当前状态 × 输入字符 → 下一状态"的表，就是一个二维数组。整个自动机可以压缩成这张表，代码则退化成"查表、赋值、循环"三句话。

0.8 符号速查与小结

符号	读法	意思
Σ	Sigma	字母表：允许出现的全部字符
ϵ	epsilon	空串：长度为 0 的字符串
$\Phi / \emptyset$	phi / 空集	什么字符串都没有的语言（注意： $\{\epsilon\}$ 不是空集，它有一个成员）
$L(r)$	r 的语言	正则表达式 r 所描述的那个字符串集合
$x \in A$	x 属于 A	x 是集合 A 的成员
$A \subseteq B$	A 包含于 B	A 的每个成员都在 B 里
$A \cup B$	A 并 B	在 A 里或在 B 里的东西
$A \times B$	A 叉乘 B	所有的有序对 (a, b) ， a 取自 A ， b 取自 B
$f: X \rightarrow Y$	f 从 X 到 Y	函数 f ，输入是 X 的成员，输出是 Y 的成员
$\mathcal{P}(S)$	S 的幂集	S 的所有子集组成的集合
A^*	A 星	把 A 的成员连接任意多次（含零次）得到的一切

本章小结

- 机器只认识数字；从机器语言到汇编再到高级语言，每上一层，就需要一个新的翻译程序。
- 字母表是有限的字符集合；字符串是有限的符号序列； ϵ 是空串，是连接运算的单位元。
- **语言 = 字符串的集合**。并、连接、闭包三种运算可以从小语言造出大语言。
- 树表达嵌套结构；递归是处理树的自然方式。
- 有限状态机：有限个状态、按输入转换、只记住当前状态。它快、省，正是扫描程序所需要的。

热身练习（不计分，但强烈建议做）

1. 设 $\Sigma = \{0, 1\}$ 。写出长度 ≤ 2 的全部字符串（提示：别忘了 ϵ ，一共 7 个）。
2. 设 $A = \{a, b\}$ ，写出 AA 和 A^* 的前 8 个成员。
3. 给“电梯门”画一个状态机：状态有“开”“关”“正在关”；输入有“按开门”“按关门”“到时”。
4. 画出表达式 $(1 + 2) * (3 - 4)$ 的树。括号在树里去哪了？

第 1 章

编译器概览

Introduction: What a Compiler Is and How It Works

这一章回答三个问题：编译器是什么、它周围有谁、它内部怎么工作。我们会跟着一条赋值语句 `a[index] = 4 + 2` 走完翻译的全过程，看它如何从一串字符变成一棵树、再变成几条机器指令。最后，我们会遇到一个哲学味十足的问题——用 C 写的 C 编译器，是怎么编译出来的？

对话：编译器只是一个“运行按钮”吗？

学生： 我一直以为 IDE 里那个绿色的三角形就是编译器。

教授： 那个按钮背后至少藏着五个程序：预处理器、编译器、汇编器、连接器、装入程序。编译器只是其中一个，而且它自己内部又分成六个阶段。

学生： 六个阶段？把代码翻译成机器码，为什么要分这么多步？

教授： 问得好。这正是本章的关键问题之一。答案和你写大程序的经验是一样的：一次只解决一个问题。先把字符切成单词，再把单词组织成结构，再检查结构的意思，最后才生成代码。每一步都有清楚的输入和输出，才可能把一个 10 万行的程序做对。

学生： 10 万行？编译器有那么大？

教授： 课件上说的是 1 万到 100 万行。GCC 的代码量以千万行计。但别怕——原理只有几页纸，而且我们有一个只有几百行的玩具编译器 TINY 可以整个读完。

1.1 为什么需要编译器

◆ 关键问题：人写的程序，机器怎么执行 ◆

人类习惯用接近数学和自然语言的记法表达算法；机器只执行编码成数字的指令。这中间的鸿沟由谁来填、怎样填？

定义

编译器（compiler）是一种计算机程序，它把用一种语言写的程序**翻译**成另一种语言的等价程序。它的输入叫**源程序**（source program），用**源语言**写成，通常是 C、C++ 这样的高级语言；它的输出叫**目标程序**（target program），用**目标语言**写成，通常是目标机器的**目标代码**（object code）或机器代码。编译器是非常复杂的程序，规模从一万行到一百万行代码不等。

课件里的例子非常典型。源程序是一行 Pascal 风格的赋值：

```
I := I0 + L * 2
```

目标程序则是一段与之**等价**的机器代码，用汇编形式写出来是：

```
MOVF  id3, R2      ; 把变量 L 装入寄存器 R2
MULF  #2.0, R2     ; R2 = R2 * 2.0
MOVF  id2, R1      ; 把变量 I0 装入 R1
ADDF  R2, R1       ; R1 = R1 + R2
MOVF  R1, id1      ; 把结果存回变量 I
```

注意几件事：一行源代码变成了五条指令；变量名 **I**、**I0**、**L** 变成了 **id1**、**id2**、**id3**（它们在**符号表**里有登记）；运算顺序遵守了“先乘后加”。这三件事分别对应本章后面要讲的代码生成、符号表和语法分析。

三层阶梯：从机器语言到高级语言

为了理解编译器为什么会出现，回顾一下程序设计语言的三级阶梯：

层次	写法 ("把 2 放进 X")	优点	缺点
机器语言 machine language	C7 06 0000 0002	机器直接执行	费时、乏味、极易出错
汇编语言 assembly language	MOV X, 2	用符号表示指令和地址，编写速度和正确率大幅提升；至今仍在使用	仍难写难读； 机器依赖 ——换一种 CPU 就要重写
高级语言 high-level language	X = 2	接近数学记法或自然语言； 与具体机器无关	需要一个程序来翻译它

最早的程序是直接用机器语言写的：程序员写下的是表示机器操作的数字码。汇编语言把指令和内存位置换成了符号形式，由**汇编器**（assembler）把符号翻译回数字。这是一次巨大的进步，但汇编语言仍然难写、难读，而且是**机器依赖的**（machine dependent）。

高级语言则更进一步：它的记法接近数学或自然语言，独立于任何特定机器。但它带来两个立刻要回答的问题，课件用一句话点了出来——**翻译可能吗？生成的目标代码高效吗？**这两个问题分别引出了编译理论（回答"可能吗"）和优化技术（回答"高效吗"）。

补充：第一台"翻译机"的怀疑者

1950 年代，当 John Backus 的团队提出要为 IBM 704 写一个 FORTRAN 编译器时，很多程序员并不相信"自动生成的代码"能比得上手写汇编。这个项目耗费了约 18 人年。它的成功证明了高级语言在实践上是可行的，也把"编译器能不能生成高效代码"这个问题变成了此后几十年编译研究的主线。

理论基础

编译器的理论基础来自一个意想不到的方向：语言学家 Noam Chomsky 对自然语言结构的研究。他按照文法的复杂程度和识别它们所需的算法，把语言分成了四类，这就是乔姆斯基层次（Chomsky hierarchy）。课件给出了完整的四类，我们把它整理成一张表：

类型	名称	规则形式	识别它的机器	在编译器中的用途
0 型	短语结构文法 / 无限制文法	任意形式	图灵机	理论上限，实践中不用
1 型	上下文有关文法	$xSy \rightarrow xAy$ （S 只有在上下文 x、y 中才能推出 A）	线性有界自动机	很少直接使用
2 型	上下文无关文法	$S \rightarrow A$ （S 无条件地推出 A）	下推自动机	描述程序的语法（第 3 章起）
3 型	正则文法	$S \rightarrow Aa$ （等价于正则表达式）	有穷自动机	描述单词的结构（第 2 章）

对编译器来说，最有用的是最下面两层。上下文无关文法（context-free grammar）描述程序的语法，由此发展出的分析问题（parsing problem）成了编译理论的标准组成部分；有限自动机和正则表达式则给出了表达程序设计语言单词结构的符号方法——这正是下一章的全部内容。第三块理论基础是优化技术（代码改进技术），它回答"高效吗"这个问题。

提示：为什么单词用 3 型、语句用 2 型

一个直觉：识别标识符只需要"记住当前读到了哪一步"（有限状态就够了）；识别 $((a+b)*c)$ 这样的括号嵌套，则必须数括号——需要一个栈。有限自动机没有栈，下推自动机有。所以单词交给正则表达式和有限自动机，语法交给上下文无关文法和下推自动机（也就是带栈的分析程序）。

1.2 编译器的亲戚们

编译器很少单独工作。从你写下源文件到程序真正跑起来，中间有一整条流水线，编译器只是其中一环。

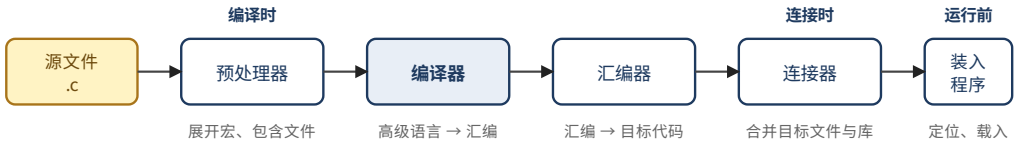


图 1.1 从源文件到可执行程序的流水线。你按下"运行"时，这五个程序依次登场。

解释器

解释器（interpreter）和编译器一样是语言翻译程序，区别在于：源程序被立即执行，而不是等整个翻译完成之后再执行。一门语言既可以被解释也可以被编译，课件把 BASIC、LISP、Java 归入解释型，把 FORTRAN、C、C++ 归入编译型。解释器和编译器共享许多操作——它同样要做词法分析、语法分析、语义分析，只是最后一步不是"生成代码"而是"直接执行"。

补充：Java 到底是编译还是解释

课件把 Java 列为解释型语言，这是教材成书年代的说法。今天的 Java 是两者的混合：javac 把源程序编译成与机器无关的字节码，JVM 再解释执行字节码，并在运行中把热点代码即时编译（JIT）成机器码。Python、JavaScript 也都走类似的混合路线。"编译"和"解释"与其说是语言的属性，不如说是实现的选择。

汇编器

汇编器（assembler）是特定计算机的汇编语言的翻译程序。由于汇编语言只是机器语言的符号形式，翻译很简单。有时编译器会生成汇编语言作为目标语言，然后由汇编器完成到目标代码的最后一步——GCC 就是这样做的。

连接器

连接器 (linker) 把分别编译或汇编得到的多个**目标文件**收集到一个最终的可执行文件里, 并把它们与标准库函数的代码、操作系统提供的资源 (内存分配器、I/O 设备等) 连接起来。历史上这曾是编译器的主要活动之一, 现在被拆分成了独立的程序。

装入程序

目标代码中的内存引用是**相对于一个尚未确定的起始位置**的——程序可能被放在内存的任何地方。**装入程序** (loader) 把所有可重定位的地址解析成相对于给定起始地址的绝对地址。装入过程通常是操作系统的一部分。

预处理器

预处理器 (preprocessor) 是编译器在翻译开始之前调用的独立程序。它可以删除注释、包含其他文件 (`#include`)、执行**宏替换** (`#define`)。宏是对重复出现的一段文本的简写。

编辑器、调试器与其他

源程序用**编辑器**写成, 产生标准的 ASCII 文件。现代编译器往往和编辑器等程序捆绑成**集成开发环境** (IDE): 编辑器懂得语言的格式, 能在你输入时报告错误, 并能从编辑器内部直接调用编译器。**调试器** (debugger) 用来找出已编译程序中的执行错误, 它需要编译器保留行号、变量名和过程名等源代码信息, 能在**断点**处暂停执行并显示调用的函数与变量的当前值。此外还有**描述器** (profiler, 统计程序各部分的执行时间) 和**项目管理程序** (如 make)。

1.3 翻译过程：六个阶段

◆ 关键问题：怎样把翻译这件大事拆成小事 ◆

从字符串到机器码的距离太远，不可能一步跨过去。编译器把翻译分成一系列阶段（phase），每个阶段接受前一阶段的输出、产生一种更接近目标的中间表示。问题是：分成哪几步？每一步的输入输出是什么？

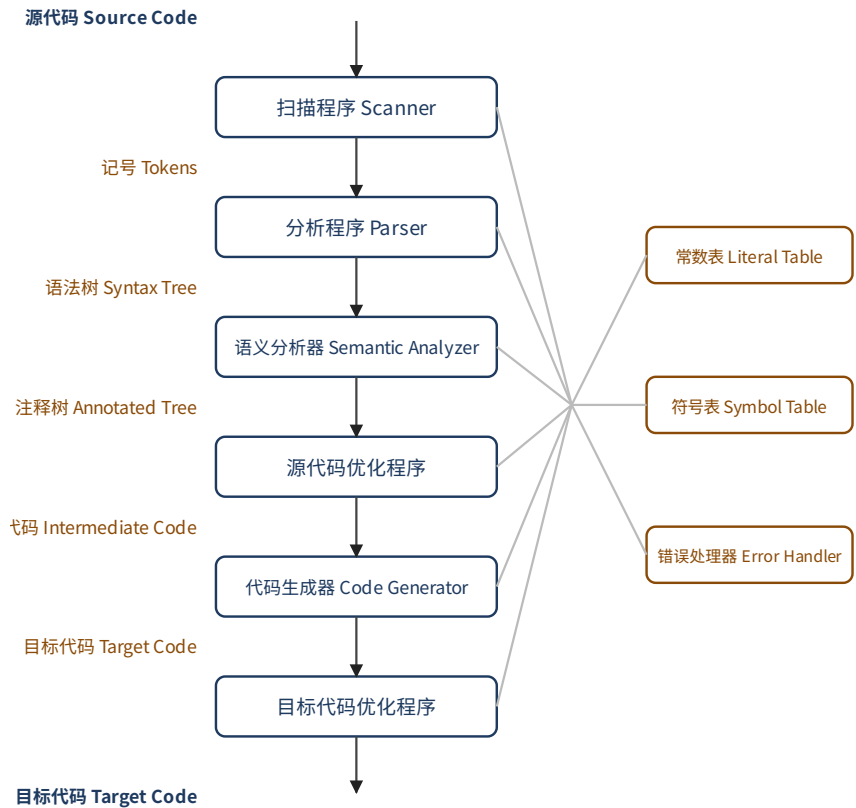


图 1.2 编译器的六个阶段与三个辅助组件。左侧标注的是每个阶段之间流动的中间表示；右侧的三个表/组件被几乎所有阶段共享。

我们用同一个例子贯穿六个阶段——课件里的 C 语句：

```
a[index] = 4 + 2
```

阶段一：扫描程序

扫描程序（scanner）读入源程序——一个字符流——执行**词法分析**（lexical analysis）：把字符序列收集成有意义的单元，称为**记号**（token），并判断是否有词法错误（例如出现了语言中不存在的字符）。我们的例子被切成 8 个记号：

记号的文本	类别	记号的文本	类别
a	标识符 identifier	=	赋值 assignment
[	左方括号 left bracket	4	数 number
index	标识符 identifier	+	加号 plus sign
]	右方括号 right bracket	2	数 number

注意扫描程序把 `index` 五个字符看成一个单元，也把空格全部丢掉了。它不关心这些记号排列得对不对——那是下一个阶段的事。扫描程序是整个第 2 章的主题。

阶段二：分析程序

分析程序（parser）接收记号形式的源代码，执行**语法分析**（syntax analysis）：确定程序的**结构**，这和给自然语言的句子做语法分析（主语、谓语、宾语）很像；同时判断是否有语法错误。分析的结果表示为**分析树**（parse tree）或**语法树**（syntax tree）。

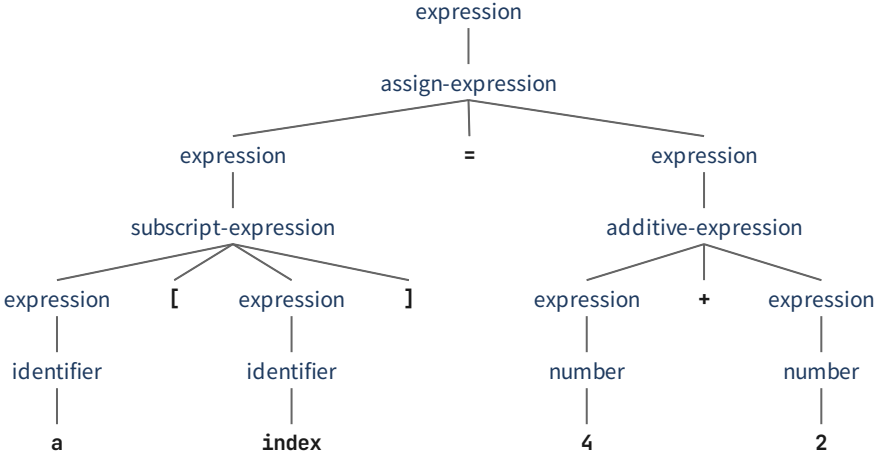


图 1.3 `a[index] = 4 + 2` 的分析树。叶子（黑色等宽字）从左到右读出来正好是原来的记号序列；内部节点（蓝色）是语法结构的名称。

分析树忠实记录了语法分析的每一步，但它太啰嗦了：`expression` → `identifier` → `a` 这一串有三层，真正的信息只有“这里是标识符 `a`”。**抽象语法树**（abstract syntax tree，简称语法树）是分析树所含信息的**浓缩**：去掉只有一个孩子的中间层，把运算符提到父节点的位置。

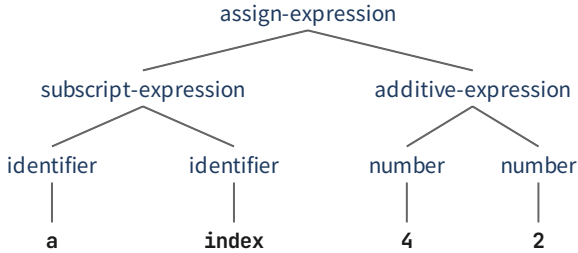


图 1.4 同一条语句的**抽象语法树**。方括号、等号、加号都不见了——它们的作用已经被树的形状表达出来。

课件在这里留了一个思考题：“如何在内存里存储树的结构？”答案在 1.4 节：每个节点是一个结构体，字段里存着指向孩子的指针。

阶段三：语义分析器

程序的**语义**（semantics）就是它的"意思"，它决定了程序运行时的行为。大多数语言有一些在**执行之前就能确定**的特性，叫**静态语义**（static semantics），典型的两项是**声明**和**类型检查**。**语义分析器**（semantic analyzer）计算出的额外信息叫**属性**（attribute），它们作为注释或"装饰"加到树上，得到**注释树**（annotated tree）。

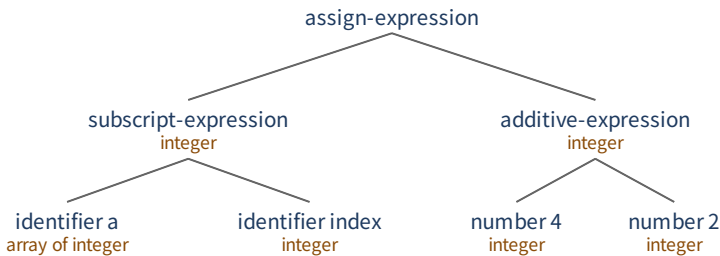


图 1.5 注释树。棕色的属性是语义分析器算出来的类型：**a** 是整数数组，用整数下标访问得到整数，等号两边类型一致——检查通过。

请体会一下语义分析器要做的推理：查符号表得知 **a** 是"整数数组"、**index** 是"整数"；数组用整数做下标是合法的，结果是"整数"；**4 + 2** 两边都是整数，和也是整数；赋值两边都是整数，合法。如果 **a** 是一个 **float** 而不是数组，语义分析器就会在这里报错。

阶段四：源代码优化程序

最早可以做优化的时刻就是语义分析之后：有些改进机会**只依赖于源代码本身**，与目标机器无关。不同编译器在优化的种类和放置的位置上差别很大。课件把这一阶段的输出叫做**中间表示**（intermediate representation, IR）或**中间代码**。

我们例子里的机会显而易见：**4 + 2** 可以由编译器预先算出来。这种优化叫**常数合并**（constant folding，也译常量折叠）。在注释树上做这件事，就是把右边

那棵子树"坍缩"成它的常量值：

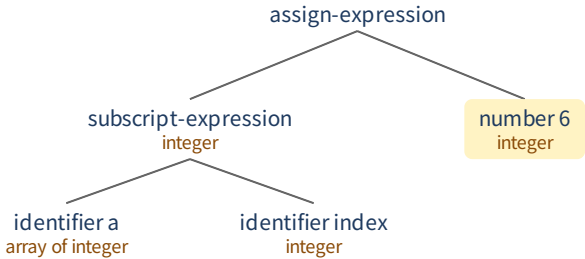


图 1.6 常数合并之后的树。整棵加法子树被一个常量 6 取代（高亮）。这一步在编译时完成，程序运行时就少做一次加法。

阶段五：代码生成器

代码生成器（code generator）接收中间代码，为目标机器生成代码。课件为了可读，把目标代码写成汇编语言；大多数真实编译器直接生成目标代码。到了这一步，**目标机器的性质变得重要起来**：要使用目标机器的指令集；要知道数据的表示方式——整数和浮点数各占几个字节或几个字。

课件的例子假设：`&a` 表示 `a` 的地址（数组的基地址）；`*R1` 表示间接寄存器寻址（以 `R1` 中的值为地址去访问内存）；机器按字节编址；整数占 2 个字节。于是 `a[index] = 6` 变成：

```

MOV  R0, index    ;; 把 index 的值装入 R0
MUL  R0, 2         ;; R0 乘以 2 (每个整数占 2 字节)
MOV  R1, &a       ;; 把 a 的地址装入 R1
ADD  R1, R0        ;; R1 = R1 + R0, 得到 a[index] 的地址
MOV  *R1, 6        ;; 把常数 6 存入 R1 所指的内存
    
```

请特别注意第二条指令：为什么要乘 2？因为 `a[index]` 的地址是“基地址 + `index` × 每个元素的大小”，而每个整数占 2 字节。这就是“目标机器的性质变得重要”的具体含义——换一台整数占 4 字节的机器，这里就得乘 4。

阶段六：目标代码优化程序

目标代码优化程序（target code optimizer）在目标代码上做改进，包括：选择更好的**寻址方式**以提高性能；用**更快的指令**代替慢指令；**消除冗余**或不必要的操作。对上面的代码，三处改进合在一起：

```
MOV  R0, index    ;; 把 index 的值装入 R0
SHL  R0            ;; 左移一位 = 乘 2，但移位比乘法快得多
MOV  &a[R0], 6     ;; 用"基址+变址"寻址方式，一条指令完成存储
```

五条指令变成了三条：乘法换成了移位（更快的指令）；地址计算与存储合并成一条带变址寻址的指令（更好的寻址方式），顺便消除了 R1 这个临时寄存器（消除冗余）。

提示：分阶段的真正理由

六个阶段各有一种输入表示和一种输出表示：字符 → 记号 → 树 → 注释树 → 中间代码 → 目标代码。每一步只需要理解相邻的两种表示。这种“一次只做一件事”的分解，是所有大型软件都在用的思想，编译器只是它最经典的范例。分阶段还带来了另一个好处，1.5 节讲前端/后端时会看到。

1.4 编译器中的主要数据结构

编译器各阶段使用的算法和支撑这些阶段的数据结构之间有很强的相互作用：算法必须以高效的方式实现，所以数据结构的选择非常重要。课件列出了六种主要的数据结构。

记号

扫描程序把字符收集成记号时，用**符号方式**表示记号——即一个枚举类型的值，该枚举类型代表源语言的记号集合。有时还需要保留字符串本身或由它推导出的其他信息：标识符记号关联的名字、数记号的值。在大多数语言里，扫描程序一

次只需要生成一个记号（**单符号先行**，single symbol lookahead），因此可以用一个全局变量保存当前记号的信息。

语法树

语法树构造成标准的**基于指针的结构**，在分析进行时动态分配。整棵树可以用一个指向根节点的变量来保存。每个节点是一个记录（结构体），其字段表示分析程序和语义分析器收集的信息——有时这些字段本身也是动态分配的。

```
typedef struct treeNode {
    struct treeNode *child[3];    /* 指向孩子的指针 */
    struct treeNode *sibling;     /* 指向兄弟的指针 */
    NodeKind    nodekind;        /* 语句、表达式..... */
    ExpType     type;            /* 语义分析器填入的属性 */
    int         lineno;
} TreeNode;
```

TINY 编译器中语法树节点的（简化）定义——回答了 1.3 节“如何存储树”的思考题

符号表

符号表（symbol table）保存与**标识符**相关的信息：函数、变量、常量和数据类型。它与编译器的几乎每个阶段都有交互：扫描程序或分析程序把标识符放进去，语义分析器加入类型信息，代码生成器查询地址。插入、删除和访问操作都必须高效，标准的数据结构是**哈希表**（hash table，课件也叫杂凑表）。

常数表

常数表（literal table）保存程序中用到的常量和字符串。快速插入和查找是关键，但不需要支持删除——一个常量整个编译过程中都可能被引用，把它放在一个地方还可以避免重复存储。

中间代码

取决于中间代码的种类和所做的优化，它可能保存为**文本字符串数组**、**临时文本文件**或**结构的链表**。

临时文件

历史上计算机的内存不足以在编译期间把整个程序保存在内存里，于是用**临时文件**保存中间步骤的产物。如今内存限制已经小得多，但编译器偶尔仍会在某些步骤生成中间文件，一个例子是**反填**（backpatching）：生成跳转指令时目标地址还不知道，先留空，等地址确定后再回头填上。

1.5 编译器结构中的其他问题

分析与综合

编译器中**分析源程序以计算其性质**的操作属于**分析**（analysis）部分：词法分析、语法分析、语义分析。产生**翻译后代码**的操作属于**综合**（synthesis）部分：代码生成。优化步骤可能同时涉及分析和综合。

前端与后端

另一种更重要的划分：**只依赖于源语言**的操作叫**前端**（front end），**只依赖于目标语言**的操作叫**后端**（back end）。这种结构对编译器的**可移植性**（portability）特别重要。

补充： $M + N$ 而不是 $M \times N$

假设你要支持 M 种源语言和 N 种目标机器。如果每个组合各写一个编译器，需要 $M \times N$ 个。但如果所有前端都产生同一种中间表示（IR），所有后端都从这种 IR 出发，那么只需要 M 个前端和 N 个后端，共 $M + N$ 个。这正是 GCC 和 LLVM 的结构：Clang、Rust、Swift 等前端都生成 LLVM IR，再由同一批后端翻译到 x86、ARM、RISC-V。中间代码就是让前端和后端“解耦”的那个接口。

遍

编译器在生成代码之前，常常要把整个源程序处理好几次，每一次叫一**遍**（pass）。遍和阶段**不一定对应**：一遍可以包含多个阶段，一个阶段也可以跨多遍。取决于语言，编译器可以是**单遍**的——编译效率高，但目标代码效率不高；Pascal 和 C 都被设计成可以单遍编译。大多数带优化的编译器使用多遍，典型的分组是：① 扫描和分析；② 语义分析和源代码级优化；③ 代码生成和目标代码优化。

语言定义

一门语言的词法、语法和语义的描述汇集在**语言参考手册**（language reference manual）或**语言定义**中。设计新语言时，语言定义和编译器往往一起开发；更常见的情形是为一门已经有定义的知名语言写编译器。

出错处理

出错处理是编译器**最重要的功能之一**。错误可能在几乎每个阶段被检测出来；编译器报告的错误都是**静态错误**（编译时错误）。生成**有意义的**错误信息很重要；错误处理器为不同的阶段和情况提供不同的操作。

提示：分辨"语法错误"和"语义错误"

少一个右括号、多一个分号、关键字拼错——分析程序能发现，是**语法错误**。类型不匹配、变量未声明、给数组整体赋一个整数——结构完全合法，但"意思"不对，是**语义错误**。作业 1.3 要求你各举两个例子，并观察你常用的编译器是先报完所有语法错误再报语义错误，还是混在一起报——后者说明它在一遍里同时做了语法分析和语义分析。

1.6 自举与移植

◆ 关键问题：编译器本身用什么语言写？ ◆

用 C 写 C 编译器，听起来像是"用一本词典的定义去解释这本词典"。第一个 C 编译器从哪里来？把一个编译器搬到一台新机器上，要重写多少？

编译器的语言

编译器的**实现语言**（或称**主机语言**，host language）当然可以是机器语言——最早的编译器就是这么写的。另一种做法是用**另一种已经有编译器的语言**来写新编译器：只要用已有的编译器编译一下新编译器，就得到了一个能运行的程序。

但如果已有的编译器运行的机器和目标机器**不同**呢？这时编译产生的是一个**交叉编译器**（cross compiler）——为另一台机器生成目标代码的编译器。为了把这些情形说清楚，我们需要一种图。

T 型图

一个用语言 H 写成、把语言 S 翻译成语言 T 的编译器，画成下面这样的**T 型图**（T-diagram）：

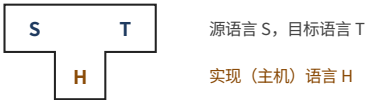


图 1.7 T 型图：横杠上写"从 S 到 T"，竖杠里写"用 H 写的"。它等价于说：这个编译器**运行**在"机器" H 上。

通常我们期望 $H = T$ ，即编译器生成的代码就是它自己所运行的那台机器的代码。T 型图的妙处在于它们可以像积木一样**拼接**，拼接的规则只有两条。

情形 1：串联两个编译器

同一台机器 H 上有两个编译器，一个把 A 翻译成 B，另一个把 B 翻译成 C。让第一个的输出成为第二个的输入，就得到了 H 上从 A 到 C 的编译器。

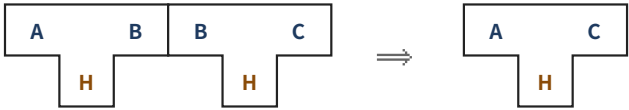


图 1.8 情形 1：两个 T 的横杠首尾相接（B 对 B），合成一个从 A 到 C 的编译器。

情形 2：用一个编译器翻译另一个编译器

更有意思的情形：我们有一个从"机器" H 到"机器" K 的编译器 M，可以用它把另一个编译器的实现语言从 H 翻译成 K。读图的方法：上面左边的 T 是被翻译的编译器（它是用 H 写的程序），下面的 T 是做翻译的编译器（源语言 H 正好对上），右边是结果——同样的编译器，但现在用 K 写成、运行在 K 上。

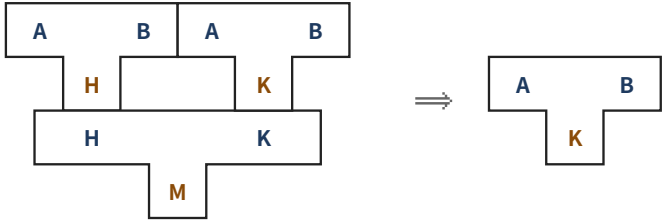


图 1.9 情形 2：左上的编译器（用 H 写）被下方的编译器（ $H \rightarrow K$ ）翻译，得到右上的编译器（用 K 写）。竖杠 H 落在下方横杠的 H 上，是这种拼接的"卡扣"。

用已有的编译器造新编译器

给定：机器 H，以及一个用 H 写的、把语言 B 翻译成 H 的编译器。想要：H 上一个把语言 A 翻译成 H 的编译器。做法：用 B 语言写 A 的编译器，然后用已有的 B 编译器编译它。这正是情形 2 的一个实例：

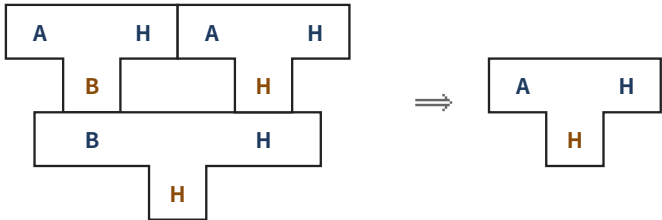


图 1.10 用 B 写一个 A 的编译器（左上），用现成的 B 编译器（下）把它编译成 H 机器码，得到 H 上可运行的 A 编译器（右）。

交叉编译器

给定：机器 K，以及一个用 K 写的、把 B 翻译成 K 的编译器。想要：语言 A 的编译器，目标语言是 H（另一台机器）。做法一样——用 B 写一个 $A \rightarrow H$ 的编译器，在 K 上编译它。结果是一个运行在 K 上、却为 H 生成代码的编译器，这就是交叉编译器。

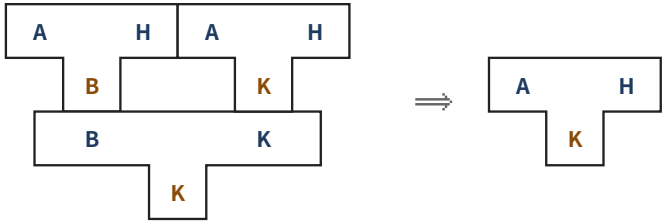


图 1.11 交叉编译器：运行在 K 上，生成 H 的代码。你在笔记本上为手机芯片编译程序，用的就是它。

H = S: 鸡生蛋的问题

现在问一个尖锐的问题：**能不能用一门语言写它自己的编译器**？比如用 C 写 C 编译器。麻烦在于：源语言的编译器还不存在，所以这个编译器本身没法被编译。这里有一个**循环**：

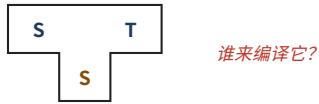


图 1.12 编译器用它要编译的语言 S 写成——但 S 的编译器正是它自己。

自举

解决办法叫**自举**（bootstrapping, "拽着自己的鞋带把自己提起来"), 分三步：

1. 用汇编语言写一个**"快而糙"**（quick and dirty）的 A 编译器——功能够用就行，生成的代码可以很差。
2. 用这个粗糙的编译器编译**"好的"**编译器（用 A 自己写的那个）。得到的编译器功能完整，但因为是粗糙编译器生成的，它本身**运行得慢**。
3. 用第 2 步得到的编译器**再编译一次**"好的"编译器。这一次，编译器的代码是由"好的"编译器生成的，于是得到了最终版本：功能完整、而且高效。

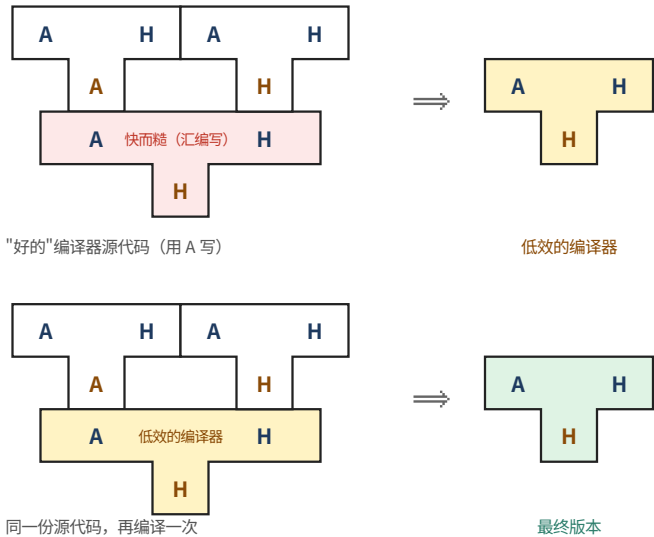


图 1.13 自举。上一行：快而糙的编译器（红）编译出一个低效但正确的编译器（黄）。下一行：用黄色的编译器再编译一次，得到最终版本（绿）。从此这门语言可以"自给自足"。

补充：自举的阴影——"信任信任"

1984 年，UNIX 之父 Ken Thompson 在图灵奖演讲《Reflections on Trusting Trust》中指出自举的一个惊人后果：如果有人在一代 C 编译器里植入一段代码，让它在编译 `login` 程序时偷偷加入后门，并且在编译**编译器本身**时把这段植入代码也复制进去——那么即使把源代码里的恶意部分删得一干二净，用被污染的编译器编译出来的新编译器仍然带毒。你读遍所有源代码也发现不了它。这个思想实验说明：**编译器是整个软件信任链的根。**

移植到新主机

自举还给了我们一个便宜的**移植**（porting）方案。假设我们有一个用 A 写的、运行在 H 上的 A 编译器，想把它搬到新机器 K 上。只需要修改编译器的**代码生成部分**，让它生成 K 的代码（这份源代码用 A 写，叫"改换目标的编译器源代码"），然后走两步：

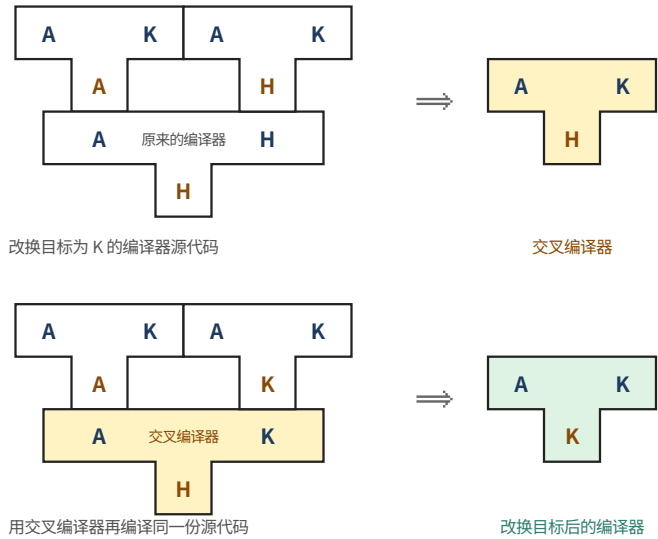


图 1.14 移植。第一步在旧机器 H 上得到一个生成 K 代码的**交叉编译器**（黄）；第二步用它再编译同一份源代码，得到**运行在 K 上的编译器**（绿）。全程只修改了代码生成部分。

提示：读 T 型图的三条口诀

- ① 横杠"从左到右"是翻译的方向，竖杠是"用什么写的 / 在哪运行"。
- ② 横杠接横杠是串联（情形 1）；竖杠落在横杠上是"被翻译"（情形 2），落下去的那个字母必须和下面横杠左端的字母一样。
- ③ 结果的横杠抄左上角的，竖杠抄下面横杠右端的。作业 1.7 就靠这三条。

1.7 TINY 语言与 TM 机器

真实机器上的真实编译器细节太多，本课程用一门小语言 **TINY** 作为贯穿全课的例子，目标代码是一台简单的假想处理器 **TM 机器** 的汇编语言。

TINY 语言

- 程序是用分号分隔的**语句序列**；没有过程，没有声明。
- 所有变量都是**整数**，通过赋值来"声明"。

- 两种控制语句：`if` 语句（可带 `else`，必须以关键字 `end` 结束）和 `repeat ... until` 语句。
- `read` 和 `write` 语句用于输入输出。
- 注释写在花括号 `{ }` 里。
- 算术表达式包含整数常量、变量、括号和四种整数运算 `+` `-` `*` `/`。
- 布尔表达式是两个算术表达式用比较运算符 `<` 或 `=` 连接而成，**只能出现在控制语句的测试位置**。
- 赋值运算符是 `:=`。

```
{ Sample program
  in TINY language -
  computes factorial
}
read x; { input an integer }
if 0 < x then { don't compute if x ≤ 0 }
  fact := 1;
  repeat
    fact := fact * x;
    x := x - 1
  until x = 0;
  write fact { output factorial of x }
end
```

计算阶乘的 TINY 程序。注意 `repeat` 循环体内最后一条语句后面没有分号——分号是**分隔符**而不是终止符。

TM 机器

TM 具有 RISC 处理器的一些特性：所有的算术和测试都必须在**寄存器**中进行；寻址方式很有限。地址的形式是“**寄存器 + 偏移量**”：`10(1)` 表示“寄存器 1 的内容加上偏移量 10”所得的地址。三条装入指令：`LDC`（load constant，装入常数）、`LD`（load，从内存装入）、`LDA`（load address，装入地址）。

课件用 `a[index] = 6` 演示 TM 代码，假设 `index` 在内存位置 10，`a` 在内存位置 20（这次每个整数占 1 个内存单元，所以乘 2 是为了演示，而不是元素大

小):

指令	作用
LDC 1,0(0)	把常数 0 装入寄存器 1 (清零, 准备当基址)
LD 0,10(1)	把地址 $10 + R1 = 10$ 处的值 (即 index) 装入 R0
LDC 1,2(0)	把常数 2 装入 R1
MUL 0,1,0	$R0 = R1 \times R0$ (index 乘 2)
LDC 1,0(0)	再次把 0 装入 R1
LDA 1,20(1)	把地址 $20 + R1 = 20$ (a 的基址) 装入 R1
ADD 0,1,0	$R0 = R1 + R0$, 得到 a[index] 的地址
LDC 1,6(0)	把常数 6 装入 R1
ST 1,0(0)	把 R1 存到地址 $0 + R0$ 处 (即 a[index])

把这段代码和 1.3 节的五条指令对照, 可以清楚看到"寻址方式有限"的代价: TM 没有 `MOV *R1, 6` 这样的间接寻址, 只能老老实实先把常数装进寄存器再存储; 连"把 0 装入寄存器"都要单独一条指令。第 8 章学代码生成时, 你会亲手为 TINY 生成这样的代码。

小结与作业

本章小结

- 编译器把源语言程序翻译成**等价**的目标语言程序。它的出现源于机器语言 → 汇编 → 高级语言的阶梯，理论基础是乔姆斯基层次（单词用 3 型、语法用 2 型）。
- 编译器的亲戚：解释器（边翻译边执行）、汇编器、连接器、装入程序、预处理器、编辑器/IDE、调试器、描述器。
- 六个阶段：扫描（字符→记号）、分析（记号→树）、语义分析（树→注释树）、源代码优化（→中间代码）、代码生成（→目标代码）、目标代码优化。三个共享组件：常数表、符号表、错误处理器。
- 主要数据结构：记号（枚举 + 属性）、语法树（指针结构）、符号表（哈希表）、常数表、中间代码、临时文件。
- 分析 vs 综合；前端 vs 后端（可移植性， $M + N$ ）；遍 $\neq$ 阶段；语言定义；出错处理。
- T 型图的两种拼接；自举三步；移植 = 改代码生成器 + 两次编译。
- TINY：无声明、全整数、if/repeat、read/write；TM：寄存器 + 偏移量寻址，LDC/LD/LDA。

作业（课件原题）

- 1.1.2 给定 C 赋值语句 `a[i+1] = a[i] + 2`，仿照本章的例子，画出该表达式的分析树和语法树。
- 2.1.3 编译错误可以粗略分为两类：**语法错误**（缺少或错放记号，如表达式 `(2+3` 中缺少右括号）和**语义错误**（表达式中的类型不正确、变量未声明等，如 `x = 2` 而 `x` 是数组变量）。
- a. 用你选择的语言，每类再各举两个例子。
 - b. 选一个你熟悉的编译器，判断它是先列出所有语法错误再列语义错误，还是两者混杂。这对遍数意味着什么？
- 3.1.7 假设你有一个用 C 写的 Pascal-to-C 翻译器和一个可用的 C 编译器。用 T 型图描述你要造出一个可用的 Pascal 编译器需要采取的步骤。

提示与参考思路见附录 B。

第 2 章

扫描：从字符到记号

Scanning: Regular Expressions and Finite Automata

编译器读到的是一个个字符；它真正想要的是一个个**单词**——标识符、数、运算符、关键字。把字符流切成单词流，就是**扫描**。这一章的结构非常清晰，只有两件事：用**正则表达式**来**描述**单词长什么样；用**有限自动机**来**识别**它们。最后再看怎样把自动机变成几十行 C 代码。

对话：扫描不就是按空格切一下吗？

学生： 把字符切成单词，用空格分一下不就行了？

教授： 试试 `a[index]=4+2`。一个空格也没有，但它有八个单词。再试试 `x≤y`：是 `≤` 一个单词，还是 `<` 和 `=` 两个？`ifx` 是关键字 `if` 加上 `x`，还是一个叫 `ifx` 的变量？

学生： ……好像每种情况都要单独写规则。

教授： 如果真的一种情况一条规则，扫描程序会变成一堆无法维护的 `if`。这一章给你两样工具：一种**语言**，让你用一行式子精确地说出"什么是一个数"；一种**机器**，把这行式子自动变成能跑的代码。它们分别叫正则表达式和有限自动机。

学生： 正则表达式我用过，在编辑器里搜索文本的那个。

教授： 正是同一个东西——它就是从编译器这里发明出来的。学完这章，你会明白编辑器为什么能在几百万行里瞬间搜到结果。

2.1 扫描的任务

扫描（scanning）或**词法分析**（lexical analysis）阶段，把源程序当作一个**字符流**读入，把它分割成一个个**记号**（token）。这是**模式匹配**（pattern matching）的一个特例，所以我们需要研究模式的**描述**和**识别**方法——正则表达式和有限自动机。此外，扫描程序是编译器中唯一逐字符处理整个源文件的部分，它必须尽可能**高效**。

记号

还是第 1 章的例子：`a[index] = 4 + 2` 被扫描程序切成 8 个记号（`a`、`[`、`index`、`]`、`=`、`4`、`+`、`2`）。记号是扫描程序生成的**逻辑单元**。课件把程序设计语言的记号分成四类：

类别	例子	说明
保留字 reserved words	<code>if</code> 、 <code>then</code> 、 <code>while</code>	拼写固定，只有一个成员
特殊符号 special symbols	<code>+</code> 、 <code>-</code> 、 <code>;</code> 、 <code>:=</code> 、 <code>≤</code>	拼写固定，可能是多个字符
数与标识符 numbers & identifiers	<code>3.14</code> 、 <code>myName</code> 、 <code>myGrade</code>	有 无穷多个 可能的拼写
字面量 literals	<code>"Hello, world!"</code> 、 <code>'a'</code>	字符串和字符常量

记号通常定义为一个**枚举类型**：

```
typedef enum { IF, THEN, ELSE, PLUS, MINUS, NUM, ID, ... } TokenType;
```

记号的属性

一个记号所代表的那串字符，叫它的**字符串值**（string value）或**词素**（lexeme）。有的记号只对应一个词素——保留字、特殊符号；有的记号可以代表

无穷多个词素——ID、NUM。凡是与记号关联的值都叫**属性** (attribute)：词素本身、NUM 记号的数值，等等。记号的属性被收集到一个结构化的数据类型——**记号记录** (token record) 里：

```
typedef struct {
    TokenType tokenval;    /* 类别: ID、NUM、PLUS,... */
    char *      stringval; /* 词素, 如 "index" */
    int         numval;    /* 若是 NUM, 它的数值 */
} TokenRecord;
```

提示：记号 ≠ 词素

记号是**类别** ("这是一个标识符"), 词素是**具体的字符串** ("index")。x 和 index 是同一个记号 ID 的两个不同词素。分析程序只看记号类别, 语义分析器才关心词素。

记号函数

扫描程序通过一个函数**按需**返回输入中的下一个记号, 同时计算它的属性:

```
TokenType getToken( );
```

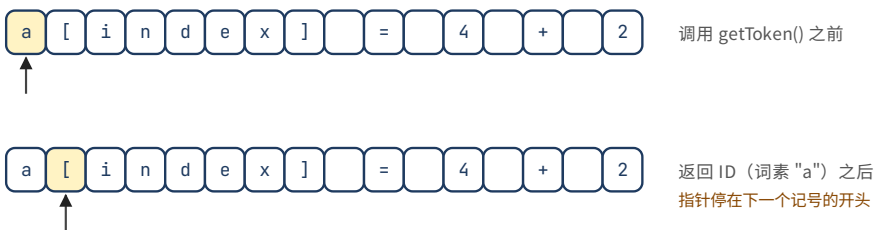


图 2.1 getToken() 每被调用一次, 就从当前位置读出一个记号并前进。分析程序需要一个记号就调一次, 扫描程序和分析程序像流水线上的两个工人一样配合。

◆ 关键问题：如何描述记号，又如何高效地识别它们 ◆

ID 有无穷多个可能的词素，不可能一一列举。我们需要一种**有限的记法**来描述**无限的集合**（这就是正则表达式），再需要一种**每读一个字符只做常数工作**的机器去识别它（这就是有限自动机）。本章其余部分就是这两件事。

2.2 正则表达式：描述模式

2.2.1 基本定义

正则表达式 (regular expression) 表示**模式** (pattern)。与一个正则表达式 r 匹配的字符串集合，叫做 r 生成的**语言**，记作 $L(r)$ 。可用的字符或符号集合叫**字母表**，记作 Σ 。正则表达式包含 Σ 中的符号，但它们指示的是模式，书写时用**粗体**；此外正则表达式还可能包含**元字符** (metacharacters) 或**元符号** (metasymbols) ——它们不是字母表的成员，而是记法本身的一部分，比如 **|**、*****、**括号**。

最基本的正则表达式只有三种：

- 字母表中的**单个字符**，它匹配自身： $r = \mathbf{a}$ ， $L(\mathbf{a}) = \{a\}$ 。
- **ϵ** 表示空串： $L(\epsilon) = \{\epsilon\}$ ——一个只含空串的集合。
- **$\emptyset$** 什么都不匹配： $L(\emptyset) = \{\}$ ——空集。

提示： **$\{\epsilon\}$ 和 $\{\}$ 不一样**

$\{\epsilon\}$ 有一个成员（空串）， $\{\}$ 一个成员都没有。前者“接受空输入”，后者“什么都不接受”。这个区别在定义 r^* 时很关键：因为 ϵ 总在 $L(r^*)$ 里，所以 $L(r^*)$ 永远不是空集。

2.2.2 三种运算

复杂的正则表达式用三种运算从简单的构造出来（按优先级从低到高排列）：

运算	元字符	写法	语言
选择 choice among alternatives		r s	$L(r s) = L(r) \cup L(s)$
连接 concatenation	(无)	r s	$L(rs) = L(r) L(s)$
重复 / 闭包 repetition, (Kleene) closure	*	r*	$L(r^*) = L(r)^*$

请把它和第 0 章的集合运算表对照：三种正则运算——**对应**三种集合运算。这就是正则表达式的全部秘密——它只是集合运算的一种紧凑写法。

选择

设 r 和 s 是正则表达式， **$r | s$** 匹配任何与 r 或 s 匹配的字符串。例子： **$a | b$** 匹配 a 或 b ， $L(a | b) = \{a, b\}$ ； **$a | \epsilon$** 匹配 a 或空串， $L(a | \epsilon) = \{a, \epsilon\}$ ； $L(a | b | c | d) = \{a, b, c, d\}$ 。

连接

ab 只匹配字符串 ab ； **$(a | b)c$** 匹配 ac 或 bc 。一般地，两个正则表达式 r 和 s 的连接写作 **rs** ，它匹配"前半与 r 匹配、后半与 s 匹配"的所有字符串。例如若 $L(r) = \{aa, b\}$ ， $L(s) = \{a, bb\}$ ，则 $L(rs) = \{aaa, aabb, ba, bbb\}$ 。

重复

r^* 匹配任意有限多个（含 0 个）与 r 匹配的字符串的连接。 **a^*** 匹配 $\epsilon, a, aa, aaa, \dots$ 。再看一个不那么平凡的例子：

$$L((a | bb)^*) = \{a, bb\}^* = \{\epsilon, a, bb, aa, abb, bba, bbbb, aaa, aabb, abba, abbbb, bbabb, bbbba, bbbbbb, \dots\}$$

每一步可以选 a 也可以选 bb ，选任意多次。注意 **$abb = a \cdot bb$** ， **$bba = bb \cdot a$** ，而 **bab 不在其中**—— b 单独出现是不允许的。

2.2.3 优先级

重复 > 连接 > 选择。因此 $a|cb^*$ 解释为 $a | (c(b^*))$ ：先算 b^* ，再和 c 连接，最后和 a 选择。想要别的意思，就像算术一样用**括号**——括号也是元字符。

补充：为什么不规定结合律？

课件的作业 2.4 问：定义里只规定了优先级，没说 $a|b|c$ 是 $(a|b)|c$ 还是 $a|(b|c)$ ，为什么？想想它们的语言： $\{a,b\} \cup \{c\}$ 和 $\{a\} \cup \{b,c\}$ 都是 $\{a,b,c\}$ 。集合的并和连接都满足结合律，所以怎么加括号**结果都一样**，自然不必规定。

2.2.4 五个例题，一次学会写正则

写正则表达式是一种技能，和写程序一样，靠例子学最快。下面五个例题都来自课件，请先自己想，再看解答。

例 2.1 $\Sigma = \{a, b, c\}$ ，恰好包含一个 b 的所有字符串

思路：这个 b 前面可以有任意多个非 b 的字符，后面也可以。"非 b 的字符"就是 $(a|c)$ ，"任意多个"就是加星号。答案：

$$(a | c)^* b (a | c)^*$$

b 不必在字符串正中间： b 、 abc 、 $abaca$ 、 $baaaac$ 都匹配。

例 2.2 $\Sigma = \{a, b, c\}$ ，至多包含一个 b 的所有字符串

"至多一个" = "恰好一个"或"零个"。用上一题的答案，再加一个"没有 b "的情况：

$$(a | c)^* | (a | c)^* b (a | c)^*$$

另一个更紧凑的解法——把 b 变成"可有可无"：

$$(a | c)^* (b | \epsilon) (a | c)^*$$

这里 ϵ 第一次显示了它的用处： $(b|\epsilon)$ 就是"一个 b 或者什么都没有"。

例 2.3 $\Sigma = \{a, b\}$ ，一个 b 两边各有相同个数的 a

$S = \{b, aba, aabaa, aaabaaa, \dots\}$ 。你可能想写 a^*ba^* ——但它也匹配 aab ，两边个数不等。事实上这个集合无法用正则表达式描述。原因可以用泵引理 (pumping lemma) 证明，直觉上是：正则表达式（等价地，有限自动机）没有办法数数——它记不住左边到底出现了多少个 a 。这就是 1.1 节说的"单词用 3 型文法、语法用 2 型文法"的分界线。

例 2.4 $\Sigma = \{a, b, c\}$ ，不含两个连续 b 的所有字符串

这题课件分了几步，展示了"逐步构造"的方法：

1. 要求每个 b 后面必须紧跟一个 a 或 c ： $(b(a|c))^*$ 。但这漏掉了单独的 a 、 c ，以及末尾的 b 。
2. 允许 a 、 c 自由出现： $((a|c)^* | (b(a|c))^*)^*$ ，化简为 $((a|c) | (ba|bc))^*$ ，即 $(a|c|ba|bc)^*$ 。
3. 字符串可以以 b 结尾（后面没有字符也不算"连续")： $(a|c|ba|bc)^* (b|\epsilon)$ 。

另一个对称的答案是把 b 放在前面看： $(b|\epsilon)(a|c|ab|cb)^*$ 。两个答案生成同一个语言——正则表达式不唯一。

例 2.5 描述 $((b|c)^* a (b|c)^* a)^* (b|c)^*$ 生成的语言

读法：括号里是"任意多个非 a ，一个 a ，任意多个非 a ，再一个 a "—— a 成对出现；整个括号重复任意次，最后再跟任意多个非 a 。所以它生成 $\Sigma = \{a, b, c\}$ 上含偶数个 a 的所有字符串（0 个也算偶数）。

提示：写正则的三招

① 先写"最典型"的字符串，再问"哪些位置可以变、怎么变"。② "可有可无"用 $(x|\epsilon)$ ；"任意多个"用星号。③ 写完后用反例检验：找一个不该匹配的字符串，看看你的式子会不会错误地接受它（例 2.3 的 aab 就是这样发现问题的）。

2.2.5 扩展记法

为了书写方便，实际使用的正则表达式增加了一些**扩展**。它们没有增加描述能力（每一种都能用三种基本运算写出来），只是更短：

扩展	写法	等价的基本形式 / 说明
一次或多次重复	<code>r+</code>	<code>r r*</code>
任意字符	<code>.</code>	字母表中任意一个字符
字符范围 / 集合	<code>[0-9]</code> 、 <code>[abc]</code>	<code>0 1 ... 9</code> 、 <code>a b c</code>
不在给定集合中的字符	<code>~a</code> 、 <code>~(a b c)</code> 、 <code>[^a]</code> 、 <code>[^abc]</code>	字母表中除这些之外的任意字符
可选子表达式	<code>r?</code>	<code>r ε</code>

课件的例子：`natural = [0-9]+`，`signedNatural = (+|-)? natural`。

2.2.6 描述真实的记号

数

```
nat      = [0-9]+
signedNat = (+|-)? nat
number   = signedNat ("." nat)? (E signedNat)?
```

请逐层读它：自然数是一串数字；带符号自然数前面可选一个正负号；一个数是"带符号自然数，可选一个小数部分，可选一个指数部分"。于是42、-3.14、+2E-5、1.5E10都是number。注意小数点写成`"."`：引号是**转义字符**，它关闭元字符的特殊含义——不加引号的`.`表示"任意字符"，加了引号才是真正的小数点。

保留字与标识符

```
reserved    = if | while | do | ...  
letter      = [a-zA-Z]  
digit       = [0-9]  
identifier  = letter ( letter | digit )*
```

标识符：以字母开头，后面跟任意多个字母或数字。`x`、`x1`、`myGrade` 合法；`1x` 不合法。

注释

注释在扫描时被**忽略**：扫描程序必须识别它们，然后丢掉。注释可能没有显式的记号——它是一个**伪记号**（pseudotoken）。两种典型的注释：

```
{Pascal comment}      {(~)}*  
--Ada comment          --(~newline)*
```

Pascal 注释：左花括号，任意多个**不是右花括号**的字符，右花括号。Ada 注释：两个减号，然后任意多个非换行字符。

2.2.7 记号定界符

记号**定界符**（delimiter）是这样一些字符：它们的出现意味着"更长的字符串不可能再是一个记号"。`xtemp=ytemp;` 中的 `=` 把两个标识符分开了；空格、制表符和换行都是定界符。要点在于：定界符**结束**了当前记号的字符串，但它**可能是下一个记号的一部分**（上例中的 `=` 本身就是一个记号）。因此定界符**不能从输入中删掉**——办法有两种：把它**退回**（backing up）输入；或者在删除之前**向前看**（looking ahead）一个字符。2.3.3 节的 `[other]` 记号就是在表达"看到了但不消耗"。

2.2.8 二义性

有些字符串能被几个不同的正则表达式匹配：`if` 和 `while` 既符合 `reserved` 又符合 `identifier`。语言定义必须给出**消除二义性的规则**。两条通行的规则：

- **关键字优先**：能解释成保留字的，就解释成保留字。
- **最长子串原则**（principle of longest substring）：能匹配多长就匹配多长。
`≤` 是一个记号而不是 `<` 和 `=`；`ifx` 是标识符而不是 `if` 加 `x`。

提示：最长子串原则解释了很多“奇怪”的报错

C 语言里 `a+++b` 被切成 `a ++ + b`（最长优先，先吃掉 `++`），所以它等于 `(a++) + b`。而 `a+++++b` 被切成 `a ++ ++ + b`，于是报错——虽然 `a++ + ++b` 是合法的，扫描程序不会“回头”去尝试别的切法。

作业（课件原题，正则表达式部分）

1. 2.1 为下列字符集写出正则表达式，或说明为什么写不出来：

- 以 `a` 开头且以 `a` 结尾的所有小写字母串；
- 以 `a` 开头或以 `a` 结尾（或两者）的所有小写字母串；
- 不含前导零的所有数字串；
- 表示偶数的所有数字串。

2. 2.2 用文字描述下列正则表达式生成的语言：

- $(a \mid b)^* a (a \mid b \mid \epsilon)$
- $(A \mid B \mid \dots \mid Z) (a \mid b \mid \dots \mid z)^*$

3. 2.4 定义正则表达式时我们规定了运算的优先级，却没有规定结合性——例如没说 `a|b|c` 是 `(a|b)|c` 还是 `a|(b|c)`，连接也是如此。为什么？

4. 2.5 证明对任意正则表达式 r ， $L(r^{**}) = L(r^*)$ 。

2.3 有限自动机：识别模式

◆ 关键问题：如何把"描述"变成"算法" ◆

正则表达式说清楚了"什么算一个标识符"，但它是一个**声明**，不是一个**过程**。我们需要一种能一步步执行、每读一个字符就能决定下一步做什么的东西。

2.3.1 什么是有限自动机

有限自动机（finite automata）或**有限状态机**（finite-state machine）是描述特定种类算法的一种**数学方式**。它可以用来**识别模式**，并且与正则表达式之间有很强的对应关系——第 0 章的闸机就是一台有限自动机。看识别标识符 `letter(letter|digit)*` 的自动机：

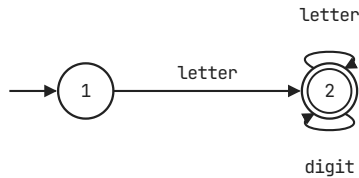


图 2.2 识别标识符的有限自动机。圆圈是**状态**；带标签的箭头是**转换**，在读到标签所示的字符时发生；没有来源的箭头指出**起始状态**；双圈是**接受状态**。

怎么用它？从状态 1 出发，逐字符读输入：读 `x`（一个 letter）走到 2；读 `1`（一个 digit）沿底部的环留在 2；输入读完，停在双圈的状态 2——**接受**，`x1` 是标识符。再试 `1x`：在状态 1 读 `1`，没有标 digit 的箭头——**拒绝**。

2.3.2 确定性有限自动机 (DFA)

把图变成数学，一个 **DFA**（deterministic finite automaton） M 由五样东西组成：

1. 一个字母表 Σ ；
2. 一个状态集合 S ；

3. 一个**转换函数** $T: S \times \Sigma \rightarrow S$ (给定"当前状态"和"读到的字符", 告诉你"下一个状态") ;
4. 一个起始状态 $s_0 \in S$;
5. 一个接受状态集合 $A \subseteq S$ 。

M 接受的语言 (language accepted), 记作 $L(M)$, 是这样的字符串 $c_1c_2\cdots c_n$ 的集合 (每个 $c_i \in \Sigma$): 存在状态 $s_1 = T(s_0, c_1)$, $s_2 = T(s_1, c_2)$, $\cdots$, $s_n = T(s_{n-1}, c_n)$, 并且 $s_n \in A$ 。用人话说: **从起始状态出发, 按字符逐一转换, 最后落在接受状态**——正是你刚才用手指做的事。"确定性" (deterministic) 的意思是: T 是一个**函数**, 每个 (状态, 字符) 组合**至多**对应一个下一状态, 永远不需要做选择。

对照图 2.2: $\Sigma = \{\text{letter}, \text{digit}\}$ (把所有字母看成一类), $S = \{1, 2\}$, $T(1, \text{letter}) = 2$, $T(2, \text{letter}) = 2$, $T(2, \text{digit}) = 2$, $s_0 = 1$, $A = \{2\}$ 。 $T(1, \text{digit})$ 没有定义——这引出下一个话题。

错误转换

严格地说, T 应该对每个 (状态, 字符) 都有定义。没画出来的转换, 按惯例都通向一个**错误状态** (error state): 一旦进入就出不来、也不接受。

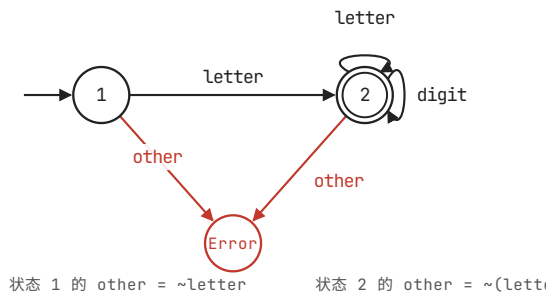
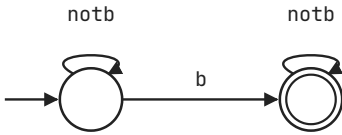


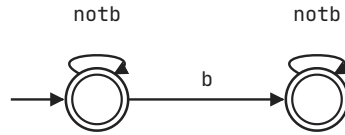
图 2.3 补上错误转换的标识符 DFA。按惯例, **错误转换不画在图上, 但总是存在。**

例 2.6 与例 2.7: 恰好一个 b / 至多一个 b

把 2.2.4 节的正则表达式画成 DFA。当不需要按名字引用状态时, 我们省略状态的标签:



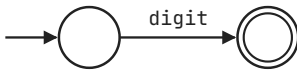
例 2.6: 恰好一个 b



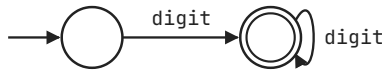
例 2.7: 至多一个 b

图 2.4 两个几乎一样的 DFA。唯一的差别：右图的起始状态**也是接受状态**——因为"零个 b"也算"至多一个"。对照 $(a|c)^*b(a|c)^*$ 与 $(a|c)^*(b|\epsilon)(a|c)^*$ 。

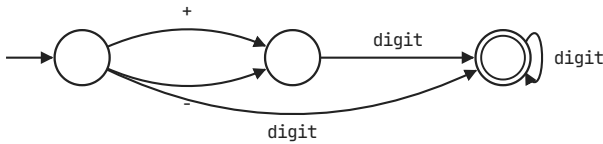
例 2.8: 数



$\text{digit} = [0-9]$



$\text{nat} = \text{digit}^+$



$\text{signedNat} = (+|-)? \text{nat}$

图 2.5 digit、nat 和 signedNat 的 DFA。注意 signedNat 里从起始状态直接读 digit 的那条弧——它就是正则表达式里的"?"：符号可以没有。

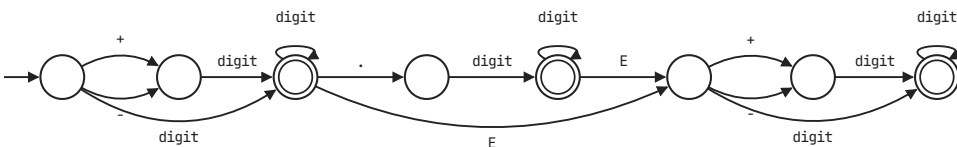


图 2.6 $\text{number} = \text{signedNat} ("." \text{nat})? (E \text{ signedNat})?$ 的 DFA。它是三个 signedNat 形状的机器串起来的，两条"绕过去"的弧分别对应两个可选部分。三个双圈说明 42、42.5、42.5E3 都可以在此结束。

例 2.9：注释

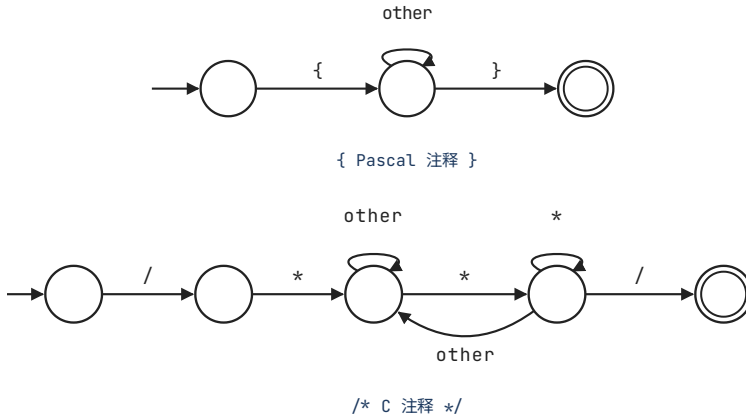


图 2.7 两种注释的 DFA。C 注释的机器值得细看：读到 * 进入"可能要结束了"的状态 4；再读 / 就结束；读到另一个 * 留在 4；读到其他字符则**退回**状态 3——"刚才那个星号只是普通内容"。

提示：给状态起"含义"

画 DFA 时，给每个状态写一句话："我在这个状态意味着到目前为止读到了什么。"C 注释的状态 3 是"在注释里"，状态 4 是"在注释里且刚读到一个*"。有了含义，转换就自然而然：在状态 4 读到 /，注释结束；读到别的，回到"在注释里"。作业 2.8 用这个方法会容易很多。

2.3.3 动作：从"接受"到"返回记号"

纯粹的 DFA 只回答"接受还是拒绝"，扫描程序还得**做事**。课件规定的**动作** (actions)：

- 每做一次转换，就把输入中的这个字符**移**到一个正在累积的字符串里——它将成为记号的词素。
- 到达接受状态时，**返回**识别出的记号。
- 到达错误状态时，生成一个错误记号，或者**回溯** (backtracking)。

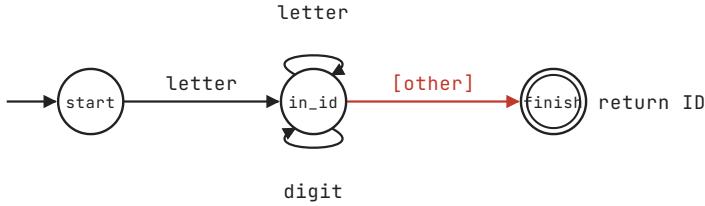


图 2.8 带动作的标识符 DFA。状态改成了有含义的名字；新增了一个 finish 状态，由 [other] 到达。**方括号表示这个定界字符应被退回输入，而不被消耗**——它属于下一个记号。

对比图 2.2：那里的自动机在状态 2 "读完就停"，而真实的扫描程序不知道输入什么时候"读完"——它必须读到一个**不属于本记号的字符**才能确定记号结束了。这就是 2.2.7 节讲的定界符，方括号 [other] 就是"看一眼、不吃掉"。

2.3.4 合并多个 DFA

一门语言有许多记号，每个记号由自己的 DFA 识别。但扫描程序只有一个入口——它需要把这些 DFA **合并**成一个。如果每个记号以**不同的字符**开头，事情很简单：把它们的起始状态合并成一个就行。课件的例子是 `:=`、`≤` 和 `=`：

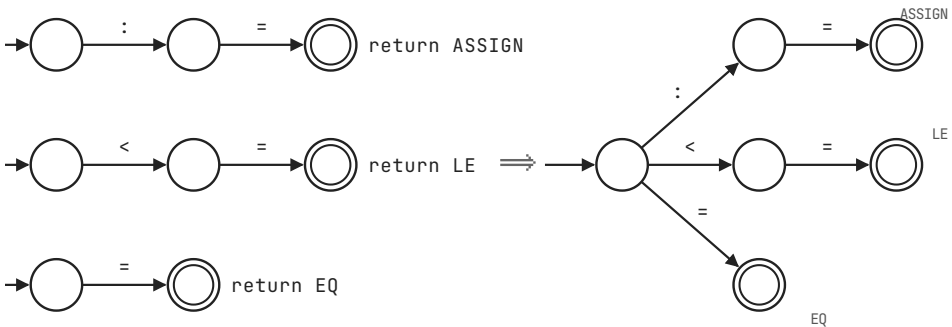


图 2.9 三个记号各有各的 DFA（左）。因为它们的首字符 `:`、`<`、`=` 互不相同，直接把三个起始状态合成一个（右），仍然是确定性的。

如果几个记号以**同一个字符**开头——比如 `<`、`≤` 和 `◇`——就不能简单合并：起始状态在读到 `<` 时会有三条出路，DFA 不允许这样。必须重新安排图，让每个状态的每个字符只有**唯一**的转换：

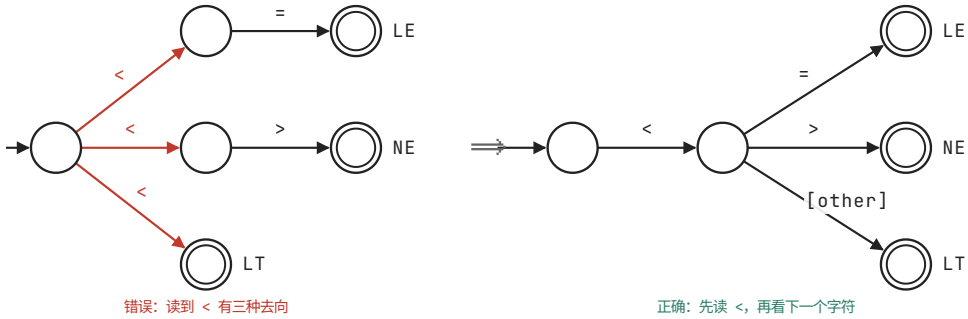


图 2.10 首字符相同的三个记号。左图的起始状态对 < 有三条出边，不是 DFA；右图把公共前缀 < 合并成一个状态，再按第二个字符分流，[other] 分支识别单独的 <（不消耗该字符）。

2.3.5 ϵ -转换与非确定性有限自动机 (NFA)

手工重排图 2.10 这样的自动机很容易，但如果有几十个记号呢？我们想要一种**机械的**合并方法——哪怕合并出来的东西暂时“不确定”，只要之后能再变回确定的就行。这需要两样新东西。

ϵ -转换

ϵ -转换 (ϵ -transition) 是一种**不看输入、不消耗任何字符**就可以发生的转换。它看起来违反直觉——转换可以“自发地” (spontaneously) 发生。但它能表达“多选一”，从而让我们**不加改动地**把多个 DFA 拼在一起：

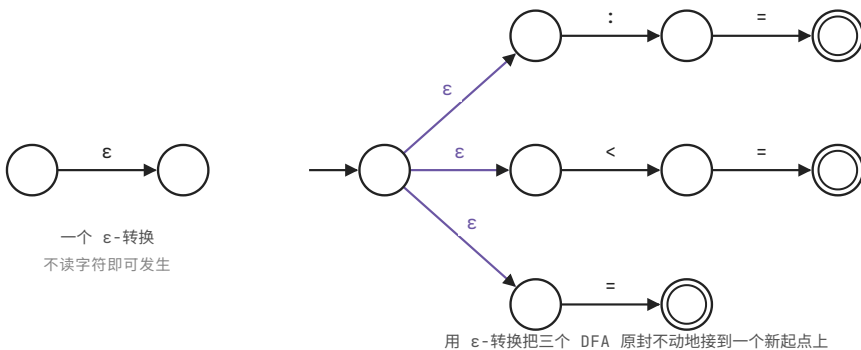


图 2.11 左：一个 ϵ -转换。右：用 ϵ -转换合并 $:$ 、 $<$ 、 $=$ 的三个 DFA——新起始状态“自发地”跳到任何一个分支。

扩展 DFA 的定义

要容纳 ϵ -转换和"一个字符多条出路", 需要修改定义中的两处:

- 字母表要包含 ϵ : $\Sigma \cup \{\epsilon\}$ 。
- 转换函数 T 的值不再是一个状态, 而是一个**状态集合**—— T 允许一个字符通向多个状态。因此 T 的值域是 S 的**幂集** (power set), 即 S 的所有子集组成的集合, 记作 $\mathcal{P}(S)$ 。

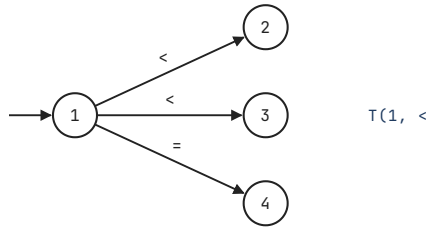


图 2.12 在状态 1 读到 $<$ 可以去 2 也可以去 3, 所以 $T(1, <)$ 是集合 $\{2, 3\}$, 而 $T(1, =) = \{4\}$ 。

NFA 的定义

一个 NFA (nondeterministic finite automaton, 非确定性有限自动机) M 由以下部分组成: 一个字母表 Σ ; 一个状态集合 S ; 一个转换函数 $T: S \times (\Sigma \cup \{\epsilon\}) \rightarrow \mathcal{P}(S)$; 一个起始状态 $s_0 \in S$; 一个接受状态集合 $A \subseteq S$ 。 M 接受的语言 $L(M)$ 是这样的字符串 $c_1c_2 \cdots c_n$ 的集合 (每个 $c_i \in \Sigma \cup \{\epsilon\}$): 存在状态 $s_1 \in T(s_0, c_1)$, $s_2 \in T(s_1, c_2)$, $\cdots$, $s_n \in T(s_{n-1}, c_n)$, 并且 $s_n \in A$ 。

和 DFA 的定义逐字对比, 只改了两处: $=$ 变成了 ϵ (下一状态是集合中的**某一个**), 字符可以是 ϵ 。课件的三条注记把含义说透了:

- c_i 中的任何一个都可以是 ϵ ; **实际被接受的字符串是去掉 ϵ 之后的那个**——所以它的长度可能比 n 小。
- 状态序列 $s_1s_2 \cdots s_n$ **不一定唯一**。
- NFA **不表示一个算法**。它可以被一个算法**模拟**: 在每个非确定性的选择处回溯尝试所有可能。

例 2.10

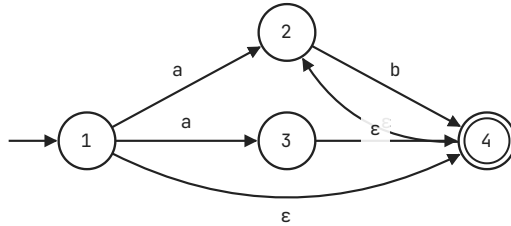


图 2.13 一个 NFA。状态 1 读 a 有两条去向；有三条 ϵ -转换。

字符串 **abb** 可以由两条不同的转换序列接受：

$1 \xrightarrow{a} 2 \xrightarrow{b} 4 \xrightarrow{\epsilon} 2 \xrightarrow{b} 4$ 或 $1 \xrightarrow{a} 3 \xrightarrow{\epsilon} 4 \xrightarrow{\epsilon} 2 \xrightarrow{b} 4 \xrightarrow{\epsilon} 2 \xrightarrow{b} 4$

这个 NFA 接受的语言是 **$ab^+ \mid ab^* \mid b^*$** ，化简后就是 **$(a \mid \epsilon) b^*$** 。同一个语言也可以由下面这个 DFA 接受：

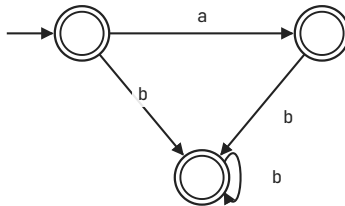


图 2.14 接受 **$(a \mid \epsilon)b^*$** 的 DFA。三个状态都是接受状态： ϵ 、a、b…、ab… 都在语言里。

补充：NFA 和 DFA 谁更强？

看起来 NFA 更"自由"，但一个著名的定理说：**每个 NFA 都存在一个接受同样语言**的 DFA（子集构造法，下一讲的内容，图 2.13 $\rightarrow$ 图 2.14 就是一例）。所以两者描述能力相同，而且都恰好等于正则表达式。区别在于用途：NFA 便于**机械地构造**（从正则表达式或多个 DFA 拼出来），DFA 便于**高效地执行**（不需要回溯）。实际的扫描程序生成器（如 Lex）走的正是"正则表达式 $\rightarrow$ NFA $\rightarrow$ DFA $\rightarrow$ 代码"这条路。

2.4 把自动机变成代码

◆ 关键问题：图纸上的 DFA 怎样变成会跑的程序 ◆

DFA 是一张图。有三种把它写成程序的方法，从"手工翻译"到"通用引擎"，各有取舍。我们用同一个 DFA 演示三种方法。

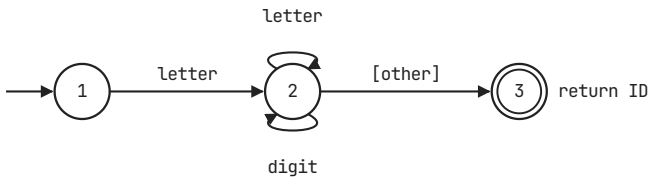


图 2.15 本节的示例 DFA：识别标识符，状态 3 为接受状态，通过 [other]（不消耗）到达。

方法一：把状态写进程序结构

最直接的写法：用程序的**控制结构**对应状态——进入 if 分支相当于换了状态，while 循环相当于自环：

```
{starting in state 1}
if the next character is a letter then
    advance the input; {now in state 2}
    while the next character is a letter or a digit do
        advance the input; {stay in state 2}
    end while;
    {go to state 3 without advancing the input}
    accept;
else
    {error or other cases}
endif
```

短小、易懂，而且很快。缺点是：状态藏在代码的**形状**里，DFA 稍微复杂一点（比如图 2.6 的 number），嵌套的 if/while 就会变得难以对应和修改。

方法二：状态变量 + 双重 switch

更好的方法：用一个变量保存当前状态，转换用循环里**双重嵌套的 switch** 实现——外层按状态分支，内层按输入字符分支：

```
state := 1; {start}
while state = 1 or state = 2 do
  switch state
  case 1: switch input character
    case letter: advance the input; state := 2; break;
    default:    state := error;
  end switch;
  case 2: switch input character
    case letter:
    case digit: advance the input; state := 2; break;
    default:    state := 3;
  end switch;
end switch;
end while;
if state = 3 then accept else error;
```

现在代码和图一一对应：每个 case state 就是一个圆圈，每个内层 case 就是一条箭头。增加状态或转换只需要增加 case，不必重排整个程序。注意 default: state := 3 分支**没有** advance the input——这就是 [other] 的"不消耗"。

方法三：转换表

既然每个内层 case 都只是"给定（状态，字符）→ 下一状态"，那么整个 DFA 可以压缩成一张**转换表**（transition table）：行是状态，列是输入字符，格子里是下一个状态。

状态 \ 输入	letter	digit	other	接受?
1	2			no
2	2	2	[3]	no
3				yes

空格子表示错误转换；[3] 的方括号仍然表示"转换但不前进输入"。有了表，代码退化成一个与具体 DFA 无关的通用循环：

```
state := 1;
ch := next input character;
while not Accept[state] and not error(state) do
    newstate := T[state][ch];
    if Advance[state][ch] then ch := next input char;
    state := newstate;
end while;
if Accept[state] then accept;
```

这里用了两个布尔数组：Advance，按状态和字符索引，指出哪些转换要前进输入（方括号的那些为 false）；Accept，按状态索引，指出哪些是接受状态。课件总结了这种方法的利弊：同样的代码可以用于许多不同的问题——换一个 DFA 只需要换表，代码一行不改；这也意味着表可以由程序自动生成（这就是 Lex/Flex 做的事）。代价是转换表可能需要很大的空间：状态数 × 字符数，对于几百个状态和 256 个字符的真实扫描程序，需要压缩技术。

方法	状态在哪里	优点	缺点
① 程序结构	藏在 if/while 的嵌套里	最短、最快	难对应、难修改
② 状态变量 + switch	一个变量，代码逐条对应	与图一一对应，易维护	仍需手写每个转换
③ 转换表	一张二维表	代码通用，表可自动生成	表可能很大

提示：三种方法其实是同一个东西

方法一把表"编译"进了控制流，方法二把表写成了 switch，方法三把表存成了数据。TINY 的扫描程序用的是方法二（手写），Flex 生成的扫描程序用的是方法三（表 + 通用循环）。理解这一点，你就能看懂任何一个扫描程序的源代码。

小结与作业

本章小结

- 扫描把字符流切成记号；记号有类别（枚举）和属性（词素、数值）；`getToken()` 按需返回下一个记号。
- 正则表达式用三种运算（选择 |、连接、闭包 *，优先级递增）从单字符、 ϵ 、 Φ 构造模式； $L(r)$ 是它描述的语言；扩展记法 $+ ? [] \sim$ 只是缩写。
- 数、标识符、保留字、注释都可以用正则表达式描述；**定界符**要退回或先行；**二义性**靠"关键字优先"和"最长子串"消除；有些集合（ $a^n b a^n$ ）无法用正则表达式描述。
- $DFA = (\Sigma, S, T: S \times \Sigma \rightarrow S, s_0, A)$ ；错误转换默认存在；带动作的 DFA 累积词素、在接受状态返回记号，`[other]` 表示看而不取。
- 首字符不同的 DFA 直接合并起点；首字符相同要重排。 ϵ -转换让合并机械化，代价是得到 NFA：T 的值是状态集合 $\mathcal{R}(S)$ ，接受路径不唯一，NFA 本身不是算法。
- 三种实现：控制结构、状态变量 + 双重 switch、转换表 + 通用循环。

作业（课件原题，有限自动机部分）

1.2.8 为练习 2.1 中 (a)–(d) 的每个字符集画出一个 DFA，或说明为什么不存在这样的 DFA。

提示与参考思路见附录 B。建议做完 2.1 之后立刻做 2.8：把你写的正则表达式画成 DFA，是检验它对不对的最好办法。

附录 A

术语中英对照表

Glossary

课件是中英双语的，考试和教材都会用到英文术语。下表按章节整理了本讲义出现的全部术语，中文译名以课件为准。

英文	中文	一句话解释
第 1 章		
compiler	编译器	把源语言程序翻译成等价目标语言程序的程序
source / target language	源语言 / 目标语言	翻译的输入语言 / 输出语言
host / implementation language	主机语言 / 实现语言	编译器本身用什么语言写
machine language	机器语言	用数字编码的指令
assembly language	汇编语言	机器语言的符号形式；机器依赖
Chomsky hierarchy	乔姆斯基层次	按文法复杂度对语言的四级分类
context-free grammar	上下文无关文法	2 型文法，描述程序语法
regular grammar	正则文法	3 型文法，等价于正则表达式
interpreter	解释器	边翻译边执行的语言处理程序
assembler / linker / loader	汇编器 / 连接器 / 装入程序	翻译汇编 / 合并目标文件 / 解析重定位地址
preprocessor	预处理器	删注释、包含文件、宏替换
debugger / profiler	调试程序 / 描述器	定位执行错误 / 统计执行时间
scanner, lexical analysis	扫描程序，词法分析	字符 → 记号

parser, syntax analysis	分析程序，语法分析	记号 → 语法树
parse tree / (abstract) syntax tree	分析树 / （抽象）语法树	完整推导记录 / 它的浓缩
semantic analyzer, static semantics	语义分析器，静态语义	声明、类型检查等执行前可判定的性质
attribute, annotated tree	属性，注释树	语义分析算出的信息及带信息的树
intermediate representation (IR)	中间表示 / 中间代码	前端与后端之间的接口
constant folding	常数合并	编译时预先计算常量表达式
code generator	代码生成器	中间代码 → 目标代码
literal table / symbol table	常数表 / 符号表	常量与字符串 / 标识符信息（哈希表）
single symbol lookahead	单符号先行	一次只需向前看一个记号
backpatch	反填	先留空后回填跳转地址
analysis / synthesis	分析 / 综合	计算源程序性质 / 产生目标代码
front end / back end	前端 / 后端	只依赖源语言 / 只依赖目标语言的部分
pass	遍	对源程序（或其表示）的一次完整处理
portability	可移植性	换机器所需改动的多少
cross compiler	交叉编译器	为另一台机器生成代码的编译器
T-diagram	T 型图	表示编译器 $S \rightarrow T$ 用 H 写的图

bootstrapping	自举	用语言自身写它的编译器的三步法
porting	移植	改代码生成器 + 两次编译
第 2 章		
token	记号	扫描程序输出的逻辑单元
lexeme / string value	词素 / 字符串值	记号对应的具体字符串
attribute, token record	属性, 记号记录	与记号关联的值及打包它们的结构体
reserved word	保留字	if、then 等固定拼写的记号
literal	字面量	字符串、字符常量
pattern matching	模式匹配	扫描是它的特例
regular expression	正则表达式	描述模式的记法
alphabet Σ	字母表	可用字符的有限集合
language $L(r)$	语言	r 匹配的字符串集合
metacharacter / metasymbol	元字符 / 元符号	记法自身的符号: $ * ()$ 等
choice / concatenation / repetition	选择 / 连接 / 重复	三种基本运算
(Kleene) closure	(Kleene) 闭包	星号运算
precedence	优先级	重复 > 连接 > 选择
pumping lemma	泵引理	证明某语言不是正则的工具
escape character	转义字符	关闭元字符特殊含义的引号

pseudotoken	伪记号	被识别但不返回的记号，如注释
delimiter	定界符	标志记号结束的字符
backing up / looking ahead	退回 / 先行	处理定界符的两种方式
ambiguity	二义性	一个串匹配多个模式
principle of longest substring	最长子串原则	能匹配多长就匹配多长
finite automaton (FA)	有限自动机	有限状态、按输入转换的机器
state / transition	状态 / 转换	圆圈 / 箭头
start state / accepting state	起始状态 / 接受状态	无源箭头所指 / 双圈
DFA	确定性有限自动机	$T: S \times \Sigma \rightarrow S$
transition function	转换函数	(状态, 字符) $\rightarrow$ 下一状态
language accepted L(M)	接受的语言	能从起点走到接受状态的所有串
error transition / error state	错误转换 / 错误状态	默认存在、不画出
action	动作	转换时累积词素、接受时返回记号
backtracking	回溯	出错或不定时退回重试
ϵ-transition	ϵ -转换	不消耗输入的转换
NFA	非确定性有限自动机	$T: S \times (\Sigma \cup \{\epsilon\}) \rightarrow \mathcal{P}(S)$
power set $\mathcal{P}(S)$	幂集	S 的所有子集
transition table	转换表	状态 $\times$ 字符 $\rightarrow$ 下一状态的二维表

附录 B

作业提示与参考思路

Hints and Sketches for the Homework

这里给的是**思路**和**检验方法**，不是标准答案——正则表达式和 DFA 都不唯一，你的答案和这里不同不代表错了。请务必先自己做，再来对照。

第 1 章的作业

1.2 $a[i+1] = a[i] + 2$ 的分析树与语法树

照抄图 1.3 的模式。分析树的根是 $\text{expression} \rightarrow \text{assign-expression}$ ，它有三个孩子：左边的 expression （下面是 $\text{subscript-expression}$ ，包含 identifier a 、 $[$ 、 expression （下面是 $\text{additive-expression}$ ：identifier i 、 $+$ 、number 1)、 $]$ ），中间的 $=$ ，右边的 expression （下面是 $\text{additive-expression}$ ：左孩子是 $\text{expression} \rightarrow \text{subscript-expression}$ (a 、 $[$ 、 i 、 $]$)，中间 $+$ ，右孩子 number 2)。语法树把所有单孩子的 expression 层和括号记号删掉：

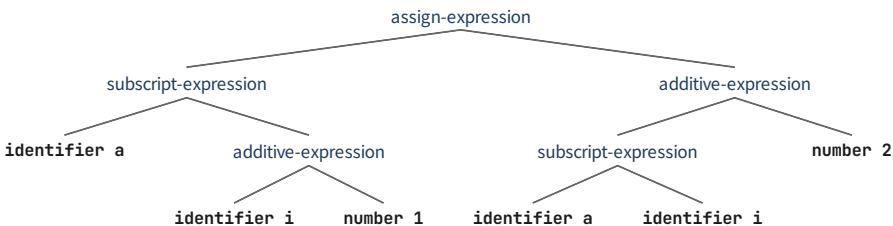


图 B.1 作业 1.2 的语法树（参考）。检验：树的叶子从左到右读出来应当是 a 、 i 、 1 、 a 、 i 、 2 ——正好是原表达式里的全部标识符和数。

1.3 语法错误与语义错误

a. 语法错误 (C 语言): `int x = 5` 缺分号; `if x > 0 { }` 条件缺括号; `for (i = 0; i < n; i++` 括号不配对。语义错误: `int x = "hello";` 类型不匹配; 使用未声明的变量 `y = 3;`; 给函数传错参数个数; `3 = x;` (左边不是左值——结构合法, 意思不合法)。

b. 用 gcc 试试一个同时含两类错误的文件。你会发现: 如果语法错误严重到分析程序无法继续, gcc 往往只报语法错误; 而在能恢复的情况下, 两类错误**混杂**出现。混杂说明它在同一遍里做语法分析和语义分析 (分析程序一边建树一边调用语义检查), 而不是先完整地做完一遍语法分析。

1.7 用 T 型图造 Pascal 编译器

已有: ① Pascal $\rightarrow$ C 的翻译器, 用 C 写 (T 型图: Pascal / C / C); ② C 编译器, 运行在机器 H 上 (C / H / H)。

1. 用 C 编译器编译翻译器 (情形 2): Pascal/C/C 落在 C/H/H 上, 得到 **Pascal / C / H**——一个能在 H 上运行的 Pascal $\rightarrow$ C 翻译器。
2. 把它和 C 编译器串联 (情形 1): Pascal/C/H 接 C/H/H, 得到 **Pascal / H / H**——一个在 H 上运行、生成 H 代码的 Pascal 编译器。

检验: 每一步都对照 1.6 节的"三条口诀", 尤其是第 1 步落下去的字母 (C) 必须等于下方横杠左端的字母 (C)。

第2章的作业

2.1 写正则表达式

设 letter = [a-z], digit = [0-9]。

- a. 以 a 开头且以 a 结尾: 别忘了长度为 1 的串 a 本身! **a | a letter* a**。

- b. 以 a 开头或以 a 结尾: $a \text{ letter}^* \mid \text{letter}^* a$ 。两部分重叠(既以 a 开头又以 a 结尾的串被两边都匹配)没有关系——选择是并集。
- c. 不含前导零的数字串: 单独的 0 是合法的, 其他串必须以 1-9 开头: $0 \mid [1-9] \text{ digit}^*$ 。
- d. 偶数: 最后一位是偶数字: $\text{digit}^* [02468]$ 。(如果也要求无前导零, 就和(c)合并: $0 \mid [1-9] \text{ digit}^* [02468] \mid [2468]$ 。)

2.2 用文字描述语言

- a. $(a|b)^* a (a|b|\epsilon)$: $\{a, b\}$ 上倒数第一位或倒数第二位是 a 的所有串。检验: ab (倒数第二位 a) $\checkmark$, ba $\checkmark$, bb $\times$, abb $\times$ 。
- b. $(A|B|\dots|Z)(a|b|\dots|z)^*$: 一个大写字母后跟任意多个小写字母——首字母大写的单词, 如 $Hello$ 、 X 。

2.4 为什么不规定结合性

因为选择和连接都满足结合律: $L((a|b)|c) = (\{a\} \cup \{b\}) \cup \{c\} = \{a\} \cup (\{b\} \cup \{c\}) = L(a|(b|c))$; $L((ab)c) = L(a(bc)) = \{abc\}$ 。既然无论怎样加括号得到的语言都一样, 就不需要规定。(对比减法: $(8-3)-2 \neq 8-(3-2)$, 所以算术必须规定减法左结合。)

2.5 证明 $L(r^{**}) = L(r^*)$

两个方向的包含。 $\supseteq$: $L(r^*) \subseteq L((r^*)^*)$ 显然——任何集合 X 都满足 $X \subseteq X^*$ (取"重复一次")。 $\subseteq$: $L(r^{**})$ 的成员是若干个 $L(r^*)$ 成员的连接; 每个 $L(r^*)$ 成员本身是若干个 $L(r)$ 成员的连接; 连接起来仍然是若干个 $L(r)$ 成员的连接, 因此属于 $L(r^*)$ 。直觉: "任意多次地做 '任意多次重复'" 还是 "任意多次重复"。

2.8 为 2.1 画 DFA

用 2.3.2 节"给状态起含义"的方法:

- a. 状态：起始、已读 a 开头且当前以 a 结尾（接受）、已读 a 开头但当前以非 a 结尾。起始读 a 到接受态；接受态读 a 留在原地、读其他字母到第三态；第三态读 a 回接受态、读其他留在原地。
- b. 状态：起始、以 a 开头（接受，之后读什么都留在这里）、不以 a 开头且当前以 a 结尾（接受）、不以 a 开头且当前不以 a 结尾。四个状态。
- c. 状态：起始、读到单独的 0 （接受，再读任何数字都出错）、读到 $1-9$ 开头（接受，读任意数字留在原地）。
- d. 状态：起始、当前末位是偶数（接受）、当前末位是奇数。每读一个数字，按它的奇偶决定去哪个状态。

四个都存在 DFA——正如 2.3.5 节的补充所说，凡是能用正则表达式写出来的，就一定有 DFA。

附录 C

自测清单

Self-Check

合上讲义，对照下面的清单逐条自问。能用自己的话回答（而不是背出课件原句）才算掌握。打不上勾的条目，回到对应小节再读一遍。

第0章

- ☐ 我能解释 ϵ 和空格的区别，以及 $\{\epsilon\}$ 和 $\{\}$ 的区别。
- ☐ 给定两个小语言 A 、 B ，我能写出 $A \cup B$ 、 AB 、 A^* 的成员。
- ☐ 我能画出一个日常设备（闸机、电梯门）的状态机，并说明"有限"指的是什么。

第1章

- ☐ 我能说出编译器的定义，并解释"等价"为什么是硬性要求。
- ☐ 我能画出机器语言 $\rightarrow$ 汇编 $\rightarrow$ 高级语言的阶梯，并说出每一层的优缺点。
- ☐ 我能列出乔姆斯基四型文法，并说出编译器用哪两型、分别用来干什么。
- ☐ 我能区分编译器和解释器、汇编器、连接器、装入程序、预处理器的职责。
- ☐ 给定一条赋值语句，我能画出它经过六个阶段时的每一种中间表示（记号、分析树、语法树、注释树、优化后的树、目标代码）。
- ☐ 我能解释常数合并、以及"目标机器的性质"是如何影响代码生成的（乘 2 那条指令）。
- ☐ 我能说出符号表和常数表的区别，以及为什么符号表用哈希表。
- ☐ 我能解释前端/后端划分为什么带来 $M + N$ 而不是 $M \times N$ 。
- ☐ 我能画出 T 型图的两种拼接，并完成作业 1.7。

- ☐ 我能用自己的话讲清楚自举的三步，以及为什么最后一步是必要的。
- ☐ 我能读懂 TINY 的阶乘程序，并解释 TM 的"寄存器 + 偏移量"寻址。

第2章

- ☐ 我能区分记号、词素、属性，并写出记号记录的结构体。
 - ☐ 我能说出正则表达式的三种基本表达式、三种运算及其优先级，并用集合运算解释每一种。
 - ☐ 给定一个用文字描述的字符串集合，我能写出正则表达式，并用反例检验它。
 - ☐ 我能解释为什么 a^nba^n 不能用正则表达式描述。
 - ☐ 我能写出 number、identifier 和两种注释的正则表达式，并解释引号转义。
 - ☐ 我能说出"最长子串原则"和"关键字优先"各解决什么二义性。
 - ☐ 我能写出 DFA 的五元组定义，并用文字解释 $L(M)$ 。
 - ☐ 给定正则表达式，我能画出 DFA；给定 DFA，我能说出它接受的语言。
 - ☐ 我能解释 [other] 的含义，以及为什么定界符不能被消耗。
 - ☐ 我能说明首字符相同/不同时如何合并 DFA，以及 ϵ -转换如何让合并机械化。
 - ☐ 我能写出 NFA 的定义，指出它与 DFA 定义的两处不同，并解释"NFA 不是算法"。
 - ☐ 我能用三种方法把一个 DFA 写成代码，并说出各自的优缺点。
-

— 第一部完 —

下一部：从有限自动机到扫描程序生成器（子集构造、DFA 最小化、Lex），以及上下文无关文法与分析。